

FPGA Verilog 代码参考手册

比赛快速查阅版

Driver 模块: 16 个文件
User 模块: 8 个文件
共计: 24 个 Verilog 文件

Driver 模块文件:

- adc_ctrl.v
- dac_ctrl.v
- ds1302_io_convert.v
- ds1302_wr_drive.v
- eeeprom_read_ctrl.v
- eeeprom_write_ctrl.v
- frequency_driver.v
- iic_drive.v
- key_ctrl.v
- led_display.v
- seg_driver.v
- spi_master.v
- sram_ctrl.v
- uart_rx.v
- uart_tx.v
- w25q128_ctrl.v

User 模块文件:

- ds1302_test.v

key_proc.v
led_proc.v
seg_proc.v
top.v
top_board.v
uart_parser.v
uart_string_sender.v

1. Driver 模块代码

1.1 adc_ctrl.v

```

module adc_ctrl (
    input      sys_clk,           // 输入: 系统时钟信号
    input      rst_n,            // 输入: 低电平复位信号
    input      ad_read_start_flag, // 输入: ADC 读取开始标志
    output     ad_read_end_flag,  // 输出: ADC 读取结束标志
    output     scl,              // 输出: I2C 时钟信号
    output [7:0] ad_data,        // 输出: ADC 转换后的数据
    inout     sda                // 双向: I2C 数据信号
);
// 参数定义
parameter P_SYS_CLK = 28'd50_000_000; // 系统时钟频率: 50MHz
parameter P_IIC_SCL = 28'd125_000;    // I2C SCL 时钟频率: 125kHz
parameter P_DEVICE_ADDR = 7'b1010_100; // I2C 从设备地址: ADC 器件地址
parameter P_ADDR_BYTE_NUM = 8'd1;     // I2C 从设备地址字节数: 1字节
parameter P_DATA_BYTE_NUM = 8'd2;     // I2C 一次操作传输数据字节数: 2字节

// 内部信号定义
wire [15:0] iic_r_data; // I2C 读取的原始数据 (16位)

// I2C 驱动模块实例化
iic_drive #(
    .P_SYS_CLK      (P_SYS_CLK), // 系统时钟频率参数
    .P_IIC_SCL      (P_IIC_SCL), // I2C SCL 时钟频率参数
    .P_DEVICE_ADDR  (P_DEVICE_ADDR), // I2C 从设备地址参数
    .P_ADDR_BYTE_NUM(P_ADDR_BYTE_NUM), // 地址字节数参数
    .P_DATA_BYTE_NUM(P_DATA_BYTE_NUM) // 数据字节数参数
) inst_dac (
    .iic_clk      (sys_clk), // 输入: I2C 时钟信号
    .iic_rst      (rst_n),   // 输入: I2C 复位信号
    .iic_start    (ad_read_start_flag), // 输入: I2C 操作开始信号 (0:空闲 1:开始)
    .iic_ready    (ad_read_end_flag),   // 输出: I2C 设备状态信号 (0:繁忙 1:空闲)
    .iic_rw_flag  (1'b1), // 输入: I2C 读写标志 (0:写 1:读)
    .iic_word_addr (0), // 输入: I2C 读写地址 (固定为0)
    .iic_wdata    (), // 输入: I2C 写数据 (先写高字节数据的高位)
    .iic_rdata    (iic_r_data), // 输出: I2C 读数据 (先读高字节数据的高位)
    .iic_rdata_valid(), // 输出: I2C 读数据有效信号 (0:无效 1:有效)
    .iic_scl      (scl), // 输出: I2C SCL 时钟信号
    .iic_sda      (sda) // 双向: I2C SDA 数据信号
);

// ADC 数据处理: 从16位原始数据中提取有效的8位数据
assign ad_data = iic_r_data[11:4]; // 取中间8位作为有效数据输出
endmodule

```

1.2 dac_ctrl.v

```

module dac_ctrl(
    input      sys_clk,           // 输入: 系统时钟信号
    input      rst_n,            // 输入: 低电平复位信号
    input      da_write_start_flag, // 输入: DAC 写入开始标志
    output     da_write_end_flag,  // 输出: DAC 写入结束标志
    output     scl,              // 输出: I2C 时钟信号
    input [7:0] da_data,        // 输入: 要写入 DAC 的数据
    inout     sda                // 双向: I2C 数据信号
);
// 参数定义
parameter P_SYS_CLK = 28'd50_000_000; // 系统时钟频率: 50MHz
parameter P_IIC_SCL = 28'd125_000;    // I2C SCL 时钟频率: 125kHz
parameter P_DEVICE_ADDR = 7'b100_1100; // I2C 从设备地址: DAC 器件地址
parameter P_ADDR_BYTE_NUM = 8'd1;     // I2C 从设备地址字节数: 1字节
parameter P_DATA_BYTE_NUM = 8'd2;     // I2C 一次操作传输数据字节数: 2字节

// 内部信号定义
wire [15:0] iic_w_data; // I2C 写入的数据 (16位)

// DAC 数据处理: 将8位数据扩展为16位, 前后各补4个0
assign iic_w_data = {4'b0000, da_data, 4'b0000}; // 数据格式转换
// I2C 驱动模块实例化

```

```

iic_drive #(
    .P_SYS_CLK      (P_SYS_CLK),      // 系统时钟频率参数
    .P_IIC_SCL      (P_IIC_SCL),      // I2C SCL 时钟频率参数
    .P_DEVICE_ADDR  (P_DEVICE_ADDR),  // I2C 从设备地址参数
    .P_ADDR_BYTE_NUM(P_ADDR_BYTE_NUM), // 地址字节数参数
    .P_DATA_BYTE_NUM(P_DATA_BYTE_NUM) // 数据字节数参数
) inst_adc (
    .iic_clk      (sys_clk),          // 输入: I2C 时钟信号
    .iic_rst      (rst_n),           // 输入: I2C 复位信号
    .iic_start     (da_write_start_flag), // 输入: I2C 操作开始信号 (0:空闲 1:开始)
    .iic_ready     (da_write_end_flag),  // 输出: I2C 设备状态信号 (0:繁忙 1:空闲)
    .iic_rw_flag   (1'b0),           // 输入: I2C 读写标志 (0:写 1:读)
    .iic_word_addr (0),              // 输入: I2C 读写地址 (固定为0)
    .iic_wdata     (iic_w_data),      // 输入: I2C 写数据 (先写高字节数据的高位)
    .iic_rdata     (),               // 输出: I2C 读数据 (先读高字节数据的高位)
    .iic_rdata_valid(),             // 输出: I2C 读数据有效信号 (0:无效 1:有效)
    .iic_scl       (scl),            // 输出: I2C SCL 时钟信号
    .iic_sda       (sda)            // 双向: I2C SDA 数据信号
);
endmodule

```

1.3 ds1302_io_convert.v

```

/*
DS1302数据传输时的 读、写 数据转换模块
1. 由于DS1302的数据手册规定数据传输有限LSB 为了对接底层的SPI的MSB优先传输的规定 故增添此模块用来 数据转换
2. 由于DS1302只有一个IO端口进行数据传输 而底层的标准SPI接口有两个数据端口 故增添此模块用来 端口转换
3. DS1302是通过置 高电平 来选择通信 与标准SPI通信协议逻辑相反
*/
module ds1302_io_convert
(
    input wire          ds1302_clk      ,
    input wire          ds1302_rst      ,
    //写
    input wire [ 7: 0]  ds1302_write_addr , //写 数据地址
    input wire [ 7: 0]  ds1302_write_data , //写 数据
    input wire          ds1302_write_en   , //写 使能
    output wire         ds1302_write_ack  , //写 完整写过程完成
    //读
    input wire [ 7: 0]  ds1302_read_addr  , //读 数据地址
    output reg [ 7: 0]  ds1302_read_data  , //读 数据
    input wire          ds1302_read_en    , //读 使能
    output wire         ds1302_read_ack   , //读 完整读过程完成
    //DS1302物理IO
    output wire         ds1302_ce         , //片选端口
    output wire         ds1302_sclk       , //SPI时钟
    inout wire          ds1302_data       //用于构建三态门
);

//状态机参数定义
localparam IDLE        = 0      ; //空闲状态
localparam CE_HIGH     = 1      ; //将CE拉高

localparam READ_ADDR   = 2      ; //发送读地址
localparam READ_DATA   = 3      ; //发送读数据

localparam WRITE_ADDR  = 4      ; //发送写地址
localparam WRITE_DATA  = 5      ; //发送写数据

localparam ACK         = 6      ; //结束 应答
localparam CE_LOW      = 7      ; //将CE拉低

// Reg define
reg [ 3: 0] state ;

reg [ 19: 0] delay_cnt ;
reg cs_ctrl ;
reg wr_req ;
reg ds1302_io_ctrl ;
reg [ 7: 0] send_data ;

wire ds1302_mosi ; //输出
wire ds1302_miso ; //输入

wire wr_ack ;
wire [ 7: 0] receive_data ;

```

```

assign          ds1302_data          = ~ds1302_io_ctrl ? ds1302_mosi      : 1'bz;
assign          ds1302_miso          = ds1302_io_ctrl ? ds1302_data      : 1'b0;

```

```

assign          ds1302_read_ack      = (state == ACK); //读完数据 应答
assign          ds1302_write_ack     = (state == ACK); //写完数据 应答

```

/******第一段状态机 状态逻辑的跳转******/

```

always@(posedge ds1302_clk or posedge ds1302_rst) begin
    if(ds1302_rst)
        state <= IDLE;
    else begin
        case(state)
            IDLE :
                if(ds1302_write_en || ds1302_read_en)
                    state <= CE_HIGH;
                else
                    state <= IDLE;
            CE_HIGH : //将CE拉高
                if(delay_cnt == 20'd255)
                    state <= ds1302_read_en ? READ_ADDR : WRITE_ADDR;
                else
                    state <= CE_HIGH;

            READ_ADDR : //发送读地址
                if(wr_ack)
                    state <= READ_DATA;
                else
                    state <= READ_ADDR;
            READ_DATA :
                if(wr_ack)
                    state <= ACK;
                else
                    state <= READ_DATA;

            WRITE_ADDR :
                if(wr_ack)
                    state <= WRITE_DATA;
                else
                    state <= WRITE_ADDR;

            WRITE_DATA :
                if(wr_ack)
                    state <= ACK;
                else
                    state <= WRITE_DATA;

            ACK : state <= CE_LOW;
            CE_LOW :
                if(delay_cnt == 20'd255)
                    state <= IDLE;
                else
                    state <= CE_LOW;
            default:state <= IDLE;
        endcase
    end
end

```

/******第二段状态机 信号操作******/

```

//delay_cnt 用于提前对CS片选进行一段时间的操作
always@(posedge ds1302_clk or posedge ds1302_rst) begin
    if(ds1302_rst)
        delay_cnt <= 'd0;
    else if(state == CE_HIGH || state == CE_LOW)
        delay_cnt <= delay_cnt + 1'b1;
    else
        delay_cnt <= 'd0 ;
end

```

```

//cs_ctrl 控制片选
always@(posedge ds1302_clk or posedge ds1302_rst) begin
    if(ds1302_rst)
        cs_ctrl <= 'd0;
    else if(state == CE_HIGH)
        cs_ctrl <= 'd1;
    else if(state == CE_LOW)
        cs_ctrl <= 'd0 ;
    else

```

```

        cs_ctrl <= cs_ctrl;
end

//wr_req 传输请求
always@(posedge ds1302_clk or posedge ds1302_rst) begin
    if(ds1302_rst)
        wr_req <= 'd0;
    else if(wr_ack)
        wr_req <= 'd0;
    else if(state == READ_ADDR || state == READ_DATA || state == WRITE_ADDR || state == WRITE_DATA)
        wr_req <= 'd1;
    else
        wr_req <= wr_req;
end

//ds1302_io_ctrl 三态门控制端口
always@(posedge ds1302_clk or posedge ds1302_rst) begin
    if(ds1302_rst)
        ds1302_io_ctrl <= 1'b0;
    else
        ds1302_io_ctrl <= (state == READ_DATA);
end

//ds1302_read_data 读取到的8位数据进行数据转换
always@(posedge ds1302_clk or posedge ds1302_rst) begin
    if(ds1302_rst)
        ds1302_read_data <= 8'h00;
    else if(state == READ_DATA && wr_ack)
        ds1302_read_data <= {receive_data[0],receive_data[1],receive_data[2],receive_data[3],
                                receive_data[4],receive_data[5],receive_data[6],receive_data[7]};
end

//send_data:发送的数据进行格式转换
always@(posedge ds1302_clk or posedge ds1302_rst) begin
    if(ds1302_rst)
        send_data <= 'd0;
    else if(state == READ_ADDR)
        send_data <= {ds1302_read_addr[0],ds1302_read_addr[1],ds1302_read_addr[2],ds1302_read_addr[3],
                        ds1302_read_addr[4],ds1302_read_addr[5],ds1302_read_addr[6],ds1302_read_addr[7]};
    else if(state == WRITE_ADDR)
        send_data <= {ds1302_write_addr[0],ds1302_write_addr[1],ds1302_write_addr[2],ds1302_write_addr[3],
                        ds1302_write_addr[4],ds1302_write_addr[5],ds1302_write_addr[6],ds1302_write_addr[7]};
    else if(state == WRITE_DATA)
        send_data <= {ds1302_write_data[0],ds1302_write_data[1],ds1302_write_data[2],ds1302_write_data[3],
                        ds1302_write_data[4],ds1302_write_data[5],ds1302_write_data[6],ds1302_write_data[7]};
end

spi_master#(
    .SYS_CLK                (28'd50_000_000        ),
    .SPI_SCLK                (28'd100_000          ),
    .SPI_CPOL                (1'b0                 ),
    .SPI_CPHA                (1'b0                 )
)
spi_master_inst(
    .spi_clk                 (ds1302_clk           ),
    .spi_rst                 (ds1302_rst           ),
    .spi_cs_ctrl             (cs_ctrl              ),// 片选控制端口
    .spi_wr_en               (wr_req               ),// 传输使能
    .spi_data_in             (send_data            ),// 数据输入
    .spi_data_out            (receive_data         ),// 数据输出
    .spi_wr_ack              (wr_ack               ),// 传输结束应答
//SPI物理端口
    .ds1302_ce               (ds1302_ce           ),// 片选端口
    .ds1302_sclk             (ds1302_sclk         ),// SPI时钟
    .spi_mosi                (ds1302_mosi         ),// 用三态门进行构建
    .spi_miso                (ds1302_miso         )
);

endmodule

```

1.4 ds1302_wr_drive.v

```

/*
DS1302完整 读、写传输 控制模块
*/
module ds1302_wr_drive(

```

```

input wire ds1302_clk ,
input wire ds1302_rst ,

//用户写数据接口
input wire [ 7: 0] write_second ,
input wire [ 7: 0] write_minute ,
input wire [ 7: 0] write_hour ,
input wire [ 7: 0] write_date ,
input wire [ 7: 0] write_month ,
input wire [ 7: 0] write_week ,
input wire [ 7: 0] write_year ,

input wire write_time_req ,//完整写操作的 写请求
output wire write_time_ack ,//完整读操作完成 应答信号

//用户读数据接口
output reg [ 7: 0] read_second ,
output reg [ 7: 0] read_minute ,
output reg [ 7: 0] read_hour ,
output reg [ 7: 0] read_date ,
output reg [ 7: 0] read_month ,
output reg [ 7: 0] read_week ,
output reg [ 7: 0] read_year ,

input wire read_time_req ,//完整读操作的 读请求
output wire read_time_ack ,//完整读操作完成 应答信号

//DS1302物理IO
output wire ds1302_ce ,//DS1302 复位引脚
output wire ds1302_sclk ,//DS1302 时钟引脚
inout wire ds1302_data //DS1302 双向数据引脚
);

//状态机 状态定义
localparam IDLE = 0 ;
//写状态
localparam WRITE_WP = 1 ;//写保护
localparam WRITE_YEAR = 2 ;
localparam WRITE_WEEK = 3 ;
localparam WRITE_MON = 4 ;
localparam WRITE_DATE = 5 ;
localparam WRITE_HOUR = 6 ;
localparam WRITE_MIN = 7 ;
localparam WRITE_SEC = 8 ;
//读状态
localparam READ_YEAR = 9 ;
localparam READ_WEEK = 10 ;
localparam READ_MON = 11 ;
localparam READ_DATE = 12 ;
localparam READ_HOUR = 13 ;
localparam READ_MIN = 14 ;
localparam READ_SEC = 15 ;
localparam ACK = 16 ;

//状态机
reg [ 4: 0] state ;
//写
reg [ 7: 0] write_addr ;//写 数据地址
reg [ 7: 0] write_data ;//写 数据
reg write_req ;//写 请求
wire write_req_ack ;//写 完整写操作 完成应答信号
//读
reg [ 7: 0] read_addr ;//读 数据地址
wire [ 7: 0] read_data ;//读 数据
reg read_req ;//读 请求
wire read_req_ack ;//读 完整读操作 完成应答信号

assign write_time_ack = (state == ACK); //完整写数据 完成 应答脉冲
assign read_time_ack = (state == ACK); //完整读数据 完整 应答脉冲

/*****第一段状态机 状态逻辑的跳转*****/

//状态机第一段 写、读数据状态机
always@(posedge ds1302_clk or posedge ds1302_rst) begin
    if(ds1302_rst)
        state <= IDLE;
    else begin
        case(state)

```

```

    IDLE      : state <= write_time_req ? WRITE_WP  :
                  read_time_req  ? READ_YEAR : IDLE ;
    WRITE_WP  : state <= write_req_ack ? WRITE_YEAR : WRITE_WP ;
    WRITE_YEAR: state <= write_req_ack ? WRITE_WEEK : WRITE_YEAR ;
    WRITE_WEEK: state <= write_req_ack ? WRITE_MON  : WRITE_WEEK ;
    WRITE_MON : state <= write_req_ack ? WRITE_DATE : WRITE_MON ;
    WRITE_DATE: state <= write_req_ack ? WRITE_HOUR : WRITE_DATE ;
    WRITE_HOUR: state <= write_req_ack ? WRITE_MIN  : WRITE_HOUR ;
    WRITE_MIN : state <= write_req_ack ? WRITE_SEC  : WRITE_MIN ;
    WRITE_SEC : state <= write_req_ack ? ACK        : WRITE_SEC ;

    READ_YEAR: state <= read_req_ack  ? READ_WEEK : READ_YEAR ;
    READ_WEEK: state <= read_req_ack  ? READ_MON  : READ_WEEK ;
    READ_MON : state <= read_req_ack  ? READ_DATE : READ_MON ;
    READ_DATE: state <= read_req_ack  ? READ_HOUR : READ_DATE ;
    READ_HOUR: state <= read_req_ack  ? READ_MIN  : READ_HOUR ;
    READ_MIN : state <= read_req_ack  ? READ_SEC  : READ_MIN ;
    READ_SEC : state <= read_req_ack  ? ACK       : READ_SEC ;

    ACK      : state <= IDLE;
    default  : state <= IDLE;
endcase
end
end

/*****第二段状态机 写状态下操作*****/

//write_req 在以下状态下 启动write操作
always@(posedge ds1302_clk or posedge ds1302_rst) begin
    if(ds1302_rst)
        write_req <= 1'b0;
    else if(write_req_ack)
        write_req <= 1'b0;
    else
        case(state)
            WRITE_WP ,
            WRITE_YEAR,
            WRITE_WEEK,
            WRITE_MON ,
            WRITE_DATE,
            WRITE_HOUR,
            WRITE_MIN ,
            WRITE_SEC : write_req <= 1'b1;
        endcase
    end
//写操作下 在不同写状态下对 地址、数据 进行赋值
always@(posedge ds1302_clk or posedge ds1302_rst) begin
    if(ds1302_rst)begin
        write_addr <= 8'h00;
        write_data <= 8'h00;
    end
    else
        case(state)
            WRITE_WP :begin write_addr <= 8'h8e; write_data <= 8'h00 ; end
            WRITE_YEAR:begin write_addr <= 8'h8c; write_data <= write_year ; end
            WRITE_WEEK:begin write_addr <= 8'h8a; write_data <= write_week ; end
            WRITE_MON :begin write_addr <= 8'h88; write_data <= write_month ; end
            WRITE_DATE:begin write_addr <= 8'h86; write_data <= write_date ; end
            WRITE_HOUR:begin write_addr <= 8'h84; write_data <= write_hour ; end
            WRITE_MIN :begin write_addr <= 8'h82; write_data <= write_minute ; end
            WRITE_SEC :begin write_addr <= 8'h80; write_data <= write_second ; end
            default :begin write_addr <= 8'h00; write_data <= 8'h00 ; end
        endcase
    end
end

/*****第二段状态机 读状态下操作*****/

//read_req 在以下状态下 启动read操作
always@(posedge ds1302_clk or posedge ds1302_rst) begin
    if(ds1302_rst)
        read_req <= 1'b0;
    else if(read_req_ack)
        read_req <= 1'b0;
    else
        case(state)
            READ_YEAR ,
            READ_WEEK ,

```



```

        READ_MON  ,
        READ_DATE ,
        READ_HOUR ,
        READ_MIN  ,
        READ_SEC   : read_req <= 1'b1;
    endcase
end

//读取数据时 需要先发送的 数据地址
always@(posedge ds1302_clk or posedge ds1302_rst) begin
    if(ds1302_rst)
        read_addr <= 8'h00;
    else
        case(state)
            READ_YEAR : read_addr <= 8'h8d;
            READ_WEEK : read_addr <= 8'h8b;
            READ_MON  : read_addr <= 8'h89;
            READ_DATE : read_addr <= 8'h87;
            READ_HOUR : read_addr <= 8'h85;
            READ_MIN  : read_addr <= 8'h83;
            READ_SEC   : read_addr <= 8'h81;
            default    : read_addr <= read_addr;
        endcase
    end
end

//在读应答时 将接收要读取的数据
always@(posedge ds1302_clk or posedge ds1302_rst) begin
    if(ds1302_rst)begin
        read_year   <= 8'h00;
        read_week   <= 8'h00;
        read_month  <= 8'h00;
        read_date   <= 8'h00;
        read_hour   <= 8'h00;
        read_minute <= 8'h00;
        read_second <= 8'h00;
    end
    else if(read_req_ack) begin
        case(state)
            READ_YEAR : read_year   <= read_data ;
            READ_WEEK : read_week   <= read_data ;
            READ_MON  : read_month  <= read_data ;
            READ_DATE : read_date   <= read_data ;
            READ_HOUR : read_hour   <= read_data ;
            READ_MIN  : read_minute <= read_data ;
            READ_SEC   : read_second <= read_data ;
        endcase
    end
end

//ds1302 module
ds1302_io_convert ds1302_io_convert_inst(
.ds1302_clk          (ds1302_clk          ),
.ds1302_rst          (ds1302_rst          ),
//写
.ds1302_write_addr    (write_addr          ),// 写 数据地址
.ds1302_write_data    (write_data          ),// 写 数据
.ds1302_write_en      (write_req           ),// 写 使能
.ds1302_write_ack     (write_req_ack       ),// 写 完整写过程完成
//读
.ds1302_read_addr     (read_addr           ),// 读 数据地址
.ds1302_read_data     (read_data           ),// 读 数据
.ds1302_read_en       (read_req            ),// 读 使能
.ds1302_read_ack      (read_req_ack        ),// 读 完整读过程完成
//DS1302物理IO
.ds1302_ce            (ds1302_ce           ),
.ds1302_sclk          (ds1302_sclk         ),
.ds1302_data          (ds1302_data         )
);

endmodule

```

1.5 eeprom_read_ctrl.v

```

module eeprom_read_ctrl(
    input sys_clk,           // 输入：系统时钟信号

```

```

    input rst_n,                // 输入: 低电平复位信号
    input eeprom_read_start_flag, // 输入: EEPROM 读取开始标志
    output eeprom_read_end_flag, // 输出: EEPROM 读取结束标志
    output scl,                 // 输出: I2C 时钟信号
    output [7:0]eeprom_read_data, // 输出: 从 EEPROM 读取的数据
    input [7:0]eeprom_read_addr, // 输入: EEPROM 读取地址
    inout sda                   // 双向: I2C 数据信号
);
// 参数定义
parameter P_SYS_CLK = 28'd50_000_000; // 系统时钟频率: 50MHz
parameter P_IIC_SCL = 28'd125_000;    // I2C SCL 时钟频率: 125kHz
parameter P_DEVICE_ADDR = 7'b101_0000; // I2C 从设备地址: EEPROM 器件地址
parameter P_ADDR_BYTE_NUM = 8'd1;     // I2C 从设备地址字节数: 1字节
parameter P_DATA_BYTE_NUM = 8'd1;     // I2C 一次操作传输数据字节数: 1字节

// I2C 驱动模块实例化
iic_drive #(
    .P_SYS_CLK      (P_SYS_CLK),        // 系统时钟频率参数
    .P_IIC_SCL      (P_IIC_SCL),        // I2C SCL 时钟频率参数
    .P_DEVICE_ADDR  (P_DEVICE_ADDR),    // I2C 从设备地址参数
    .P_ADDR_BYTE_NUM(P_ADDR_BYTE_NUM), // 地址字节数参数
    .P_DATA_BYTE_NUM(P_DATA_BYTE_NUM) // 数据字节数参数
) inst_adc (
    .iic_clk      (sys_clk),            // 输入: I2C 时钟信号
    .iic_rst      (rst_n),              // 输入: I2C 复位信号
    .iic_start    (eeprom_read_start_flag), // 输入: I2C 操作开始信号 (0:空闲 1:开始)
    .iic_ready    (eeprom_read_end_flag), // 输出: I2C 设备状态信号 (0:繁忙 1:空闲)
    .iic_rw_flag  (1'b1),              // 输入: I2C 读写标志 (0:写 1:读)
    .iic_word_addr(eeprom_read_addr),    // 输入: I2C 读写地址
    .iic_wdata    (),                  // 输入: I2C 写数据 (先写高字节数据的高位)
    .iic_rdata    (eeprom_read_data),    // 输出: I2C 读数据 (先读高字节数据的高位)
    .iic_rdata_valid(),               // 输出: I2C 读数据有效信号 (0:无效 1:有效)
    .iic_scl      (scl),               // 输出: I2C SCL 时钟信号
    .iic_sda      (sda)                // 双向: I2C SDA 数据信号
);
endmodule

```

1.6 eeprom_write_ctrl.v

```

module eeprom_write_ctrl(
    input sys_clk,                // 输入: 系统时钟信号
    input rst_n,                 // 输入: 低电平复位信号
    input eeprom_write_start_flag, // 输入: EEPROM 写入开始标志
    output eeprom_write_end_flag, // 输出: EEPROM 写入结束标志
    output scl,                  // 输出: I2C 时钟信号
    input [7:0]eeprom_write_data, // 输入: 要写入 EEPROM 的数据
    input [7:0]eeprom_write_addr, // 输入: EEPROM 写入地址
    inout sda                    // 双向: I2C 数据信号
);
// 参数定义
parameter P_SYS_CLK = 28'd50_000_000; // 系统时钟频率: 50MHz
parameter P_IIC_SCL = 28'd125_000;    // I2C SCL 时钟频率: 125kHz
parameter P_DEVICE_ADDR = 7'b101_0000; // I2C 从设备地址: EEPROM 器件地址
parameter P_ADDR_BYTE_NUM = 8'd1;     // I2C 从设备地址字节数: 1字节
parameter P_DATA_BYTE_NUM = 8'd1;     // I2C 一次操作传输数据字节数: 1字节

// I2C 驱动模块实例化
iic_drive #(
    .P_SYS_CLK      (P_SYS_CLK),        // 系统时钟频率参数
    .P_IIC_SCL      (P_IIC_SCL),        // I2C SCL 时钟频率参数
    .P_DEVICE_ADDR  (P_DEVICE_ADDR),    // I2C 从设备地址参数
    .P_ADDR_BYTE_NUM(P_ADDR_BYTE_NUM), // 地址字节数参数
    .P_DATA_BYTE_NUM(P_DATA_BYTE_NUM) // 数据字节数参数
) inst_adc (
    .iic_clk      (sys_clk),            // 输入: I2C 时钟信号
    .iic_rst      (rst_n),              // 输入: I2C 复位信号
    .iic_start    (eeprom_write_start_flag), // 输入: I2C 操作开始信号 (0:空闲 1:开始)
    .iic_ready    (eeprom_write_end_flag), // 输出: I2C 设备状态信号 (0:繁忙 1:空闲)
    .iic_rw_flag  (1'b0),              // 输入: I2C 读写标志 (0:写 1:读)
    .iic_word_addr(eeprom_write_addr),    // 输入: I2C 读写地址
    .iic_wdata    (eeprom_write_data),    // 输入: I2C 写数据 (先写高字节数据的高位)
    .iic_rdata    (),                  // 输出: I2C 读数据 (先读高字节数据的高位)
    .iic_rdata_valid(),               // 输出: I2C 读数据有效信号 (0:无效 1:有效)
    .iic_scl      (scl),               // 输出: I2C SCL 时钟信号
    .iic_sda      (sda)                // 双向: I2C SDA 数据信号
);
endmodule

```

1.7 frequency_driver.v

```

module frequency_divider (
    input wire      clk,      // 输入时钟
    input wire      rst_n,    // 低电平复位
    input wire [31:0] clkbases, // 基准时钟频率 (单位: Hz), 例如50MHz
    input wire [31:0] clkdiv,  // 期望输出频率 (单位: Hz), 如1000代表1kHz
    output reg      clkout    // 输出时钟
);
    reg [31:0] counter; // 计数器
    reg [31:0] limit;   // 计数限制值

    // 计算每个分频周期的计数限制
    always @* begin
        if (clkdiv > 0) begin
            limit = clkbases / (2 * clkdiv); // 计算分频周期
        end else begin
            limit = 0; // 防止除以零
        end
    end

    // 计数逻辑
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            counter <= 0; // 复位计数器
            clkout <= 0; // 复位输出时钟
        end else begin
            if (limit > 0) begin
                if (counter >= limit - 1) begin
                    clkout <= ~clkout; // 翻转输出时钟
                    counter <= 0; // 计数器归零
                end else begin
                    counter <= counter + 1; // 计数器加一
                end
            end
        end
    end
endmodule

```

1.8 iic_drive.v

/* IIC多字节读写 底层驱动模块

一. 模块功能:

1. 支持操作数据寄存器地址长度可变 (支持不发送器件地址)
2. 支持传输数据字节数长度可变
3. 支持突发页读, 写操作

二. 模块说明:

1. 写、读操作数据传输的顺序 高字节->低字节 高位->低位
2. 参数定义的IIC_SCL速率 用户可按照IIC器件数据手册声明
3. 突发页读、写操作时 参数定义的DATA_BYTE_NUM 不可超过器件的页字节数

三. 其他附录:

1. IIC协议速率规定:

标准模式 100kb/s
快速模式 400kb/s
增强模式 1Mb/s
高速模式 3.4Mb/s
极速模式 5Mb/s

2. AT24C02 是 2Kbit (256 字节) 的 EEPROM, 内部划分为 32 页, 每页 8 字节

3. EEPROM(电可擦除可编程只读存储器)在写入前需要进行擦写 (AT24C02 的写入操作无需用户主动擦除)

*/

```

module iic_drive
#(
    parameter P_SYS_CLK      = 28'd50_000_000, //系统时钟频率
    parameter P_IIC_SCL      = 28'd125_000,   //IIC_SCL时钟频率
    parameter P_DEVICE_ADDR  = 7'b1010_100,   //IIC从设备 器件地址(电位器器件地址:1010_100)
    parameter P_ADDR_BYTE_NUM = 8'd1,         //IIC从设备 数据地址字节数
    parameter P_DATA_BYTE_NUM = 8'd1          //IIC 1 次操作 传输数据字节数
)
(
    input iic_clk,
    input iic_rst

```

```

//IIC传输启动
input          iic_start          , //读写操作开始信号    0:    1:开始
output reg     iic_ready          , //设备忙、闲指示信号  0:繁忙  1:空闲
input          iic_rw_flag        , //读写标志信号      0:写    1:读

//IIC写数据
input          [P_ADDR_BYTE_NUM*8 -1 :0] iic_word_addr , //读写的数据地址
input          [P_DATA_BYTE_NUM*8 -1 :0] iic_wdata    , //写的数据 (先写高字节数据的高位)

//IIC读数据
output reg     [P_DATA_BYTE_NUM*8 -1 :0] iic_rdata    , //读的数据 (先读高字节数据的高位)
output reg     iic_rdata_valid    , //读数据有效信号    0:无效  1:有效

//IIC操作失败
output reg     iic_ack_error      , //应答失败信号      0:应答正常 1:应答失败

//IIC物理IO
output reg     iic_scl            , //IIC SCL
inout          iic_sda            , //IIC SDA
);

/*****激活，忙碌空闲信号*****/
wire          active              ; //传输激活信号
/*****读写控制、数据地址、写入数据 寄存*****/
reg           iic_rw_flag_reg     ; //读写控制位 寄存
reg [P_ADDR_BYTE_NUM*8 -1 :0] word_addr_reg ; //数据地址 寄存
reg [P_DATA_BYTE_NUM*8 -1 :0] wdata_reg   ; //写的数据 寄存
/*****状态机编码 状态机跳转条件组合逻辑 *****/
reg [6:0] iic_state              ; //状态机状态
//状态机状态编码（独热码）
localparam IDLE = 7'b0000_001 ; //空闲
localparam START_DEVICE_ADDR = 7'b0000_010 ; //起始位 器件地址 写标志
localparam W_WORD_ADDR = 7'b0000_100 ; //写数据地址
localparam W_DATA = 7'b0001_000 ; //写数据
localparam R_START_DEVICE_ADDR = 7'b0010_000 ; //起始位 器件地址 读标志
localparam R_DATA = 7'b0100_000 ; //读数据
localparam STOP = 7'b1000_000 ; //结束
/*****辅助SCL时钟生成 & 辅助SDA数据更新与读写 的重要计数器，标志信号 *****(和SCL时钟相关)*****/
localparam SCL_CNT_MAX = P_SYS_CLK / P_IIC_SCL - 1'b1; //计算计数器cnt_scl的最大值
reg [11:0] cnt_scl ; //分频计数器 用于生成 IIC_SCL信号

reg scl_nedge_flag ; //SCL下降沿
reg scl_pedge_flag ; //SCL上升沿
reg w_sda_flag ; //SDA写标志
reg r_sda_flag ; //SDA读标志

/*****发送数据位计数器 & 更新每个状态下数据位的最大值*****(和SDA数据相关)*****/
reg [3:0] bit_cnt ; //传输的bit计数器
reg [3:0] bit_cnt_num ; //传输的bit计数器 最大值
wire end_bit_flag ; //传输最后一位完成指示信号
//最后一位传输完成 指示
assign end_bit_flag = (bit_cnt == bit_cnt_num - 1) && (cnt_scl == SCL_CNT_MAX);

/*****发送字节计数器 & 更新每个状态下字节数最大值*****(用于解决 数据地址，读写传输数据，字节数可变的问题)*****/
reg [7:0] byte_cnt ; //传输的byte计数器
reg [7:0] byte_cnt_num ; //传输的byte计数器 最大值
wire add_byte_flag ; //多字节传输状态 字节加 指示信号
wire end_byte_flag ; //多字节传输状态 数据传输完成 指示信号

//多字节传输状态 字节加 指示
assign add_byte_flag = ((iic_state == W_WORD_ADDR)|| (iic_state == W_DATA) || (iic_state == R_DATA))&& (end_bit_flag);
//多字节传输状态 数据传输完成 指示
assign end_byte_flag = (add_byte_flag) && (byte_cnt == byte_cnt_num - 1'b1) ;
/*****IIC-SDA数据的切换与采集 *****/
wire [8:0] start_device_write ; // 起始位 器件地址 写位
wire [8:0] start_device_read ; // 起始位 器件地址 读位
//
assign start_device_write = {1'b0,P_DEVICE_ADDR,1'b0}; //用于 开始 器件地址 写传输
assign start_device_read = {1'b0,P_DEVICE_ADDR,1'b1}; //用于 开始 器件地址 读传输
/*****SDA输出，输入模式 切换*****/
reg sda_ctrl ; //三态门 控制端口 1输出 0输入
reg sda_out ; //三态门 输出
wire sda_in ; //三态门 输入
//SDA端口 三态门实现
assign sda_in = !sda_ctrl ? iic_sda : 1'b1 ; //三态门输入
assign iic_sda = sda_ctrl ? sda_out : 1'bz ; //三态门输出
/*****主机接收从机数据*****/
reg [P_DATA_BYTE_NUM*8 -1 :0] rdata_reg ; //读的数据
reg rdata_valid_reg ; //读数据有效信号 高电平有效

/*****激活，忙碌空闲信号*****/
assign active = iic_start && iic_ready; //外部输入开始 && 设备空闲 激活开始传输操作
//iic_ready 模块空闲忙碌指示信号

```

```

always@(posedge iic_clk or posedge iic_rst)begin
    if(iic_rst)
        iic_ready <= 1'b1;      //复位 空闲
    else if(active)
        iic_ready <= 1'b0;      //接收到 激活信号 忙碌
    else if(iic_state == IDLE) //回到空闲状态
        iic_ready <= 1'b1;
    else
        iic_ready <= iic_ready;
end

/*****读写控制、数据地址、写入数据 寄存*****/

//开始信号有效时，寄存输入端口的数据
always@(posedge iic_clk or posedge iic_rst)begin
    if(iic_rst)begin
        iic_rw_flag_reg    <= 'd0;
        word_addr_reg      <= 'd0;
        wdata_reg          <= 'd0;
    end
    else if(active)begin
        iic_rw_flag_reg    <= iic_rw_flag ; //读写控制位 寄存
        word_addr_reg      <= iic_word_addr ; //数据地址 寄存
        wdata_reg          <= iic_wdata ; //写的数据 寄存
    end
    else begin
        iic_rw_flag_reg    <= iic_rw_flag_reg ;
        word_addr_reg      <= word_addr_reg ;
        wdata_reg          <= wdata_reg ;
    end
end

/***** 状态机编码 状态机跳转条件组合逻辑 *****/

//二段式状态机 第一段状态跳转
always@(posedge iic_clk or posedge iic_rst)begin
    if(iic_rst)begin
        iic_state <= IDLE;
    end
    else begin
        case(iic_state)
            IDLE :begin
                if(active) //激活信号有效时，跳转到 发送 起始位 + 写器件地址 + 写标志位 状态
                    iic_state <= START_DEVICE_ADDR;
                else
                    iic_state <= iic_state;
            end
            START_DEVICE_ADDR:begin
                if( (end_bit_flag) && (P_ADDR_BYTE_NUM != 0)) //起始位 + 写器件地址 + 写标志位(10bit) 发送完成后，跳转到 写数据地址 状态
                    iic_state <= W_WORD_ADDR;
                else if((end_bit_flag) && (P_ADDR_BYTE_NUM == 0))
                    iic_state <= W_DATA;
                else
                    iic_state <= iic_state;
            end
            W_WORD_ADDR:begin
                if( (end_byte_flag) && (iic_rw_flag_reg == 1'b0) ) //数据地址(8bit) 发送完成后 如果是写操作， 跳转到 写数据 状态
                    iic_state <= W_DATA;
                else if( (end_byte_flag) && (iic_rw_flag_reg == 1'b1) ) //数据地址 发送完成后 如果是读操作，跳转到 起始位 + 写器件地址 + 读标志位 状态
                    iic_state <= R_START_DEVICE_ADDR;
                else
                    iic_state <= iic_state;
            end
            W_DATA:begin
                if( end_byte_flag ) //数据 全部发送完成后，跳到停止状态
                    iic_state <= STOP;
                else
                    iic_state <= iic_state;
            end
            R_START_DEVICE_ADDR:begin
                if( end_bit_flag ) //起始位 + 器件地址 + 读指示(共10bit) 写入完成，跳转到 读数据 状态
                    iic_state <= R_DATA;
                else
                    iic_state <= iic_state;
            end
            R_DATA:begin
                if( end_byte_flag )
                    iic_state <= STOP; //所有数据读取完成，跳转到 停止 状态
                else

```

```

        iic_state <= iic_state;
    end
    STOP:begin
        if(cnt_scl == SCL_CNT_MAX)                //停止位(1bit) 发送完成后, 跳转到 空闲 状态
            iic_state <= IDLE;
        else
            iic_state <= iic_state;
        end
        default:iic_state <= IDLE;
    endcase
end
end

always@(posedge iic_clk or posedge iic_rst)begin
    if(iic_rst)
        cnt_scl <= 'd0;
    else if((iic_state == IDLE))    //空闲状态 计数器值为零
        cnt_scl <= 'd0;
    else if(cnt_scl == SCL_CNT_MAX) //不是空闲状态时 计数器计到最大值 清零
        cnt_scl <= 'd0;
    else
        cnt_scl <= cnt_scl + 1'b1;
end

//根据cnt_scl计数器的值 生成各种标志信号
always@(posedge iic_clk or posedge iic_rst)begin
    if(iic_rst)begin
        scl_nedge_flag <= 'd0;
        scl_pedge_flag <= 'd0;
        w_sda_flag      <= 'd0;
        r_sda_flag      <= 'd0;
    end
    else if (iic_state == IDLE)begin
        scl_nedge_flag <= 'd0;    //cnt_scl计数到 0 时 SCL拉低
        scl_pedge_flag <= 'd0;    //cnt_scl计数到一半时 SCL拉高
        w_sda_flag      <= 'd0;    //cnt_scl计数到 1/4 处 SDA 写入数据
        r_sda_flag      <= 'd0;    //cnt_scl计数到 3/4 处 SDA 读取数据
    end
    else begin
        scl_nedge_flag <= (cnt_scl == 0                ); //cnt_scl计数到 0 时 SCL拉低
        scl_pedge_flag <= (cnt_scl == SCL_CNT_MAX / 2    ); //cnt_scl计数到一半时 SCL拉高
        w_sda_flag      <= (cnt_scl == SCL_CNT_MAX / 4    ); //cnt_scl计数到 1/4 处 SDA 写入数据
        r_sda_flag      <= (cnt_scl == SCL_CNT_MAX * 3 / 4 ); //cnt_scl计数到 3/4 处 SDA 读取数据
    end
end

//*****数据位计数器 & 更新每个状态下数据位的最大值***** (和SDA数据相关) *****/
//bit_cnt 数据位计数器 , 当分频计数器计数结束时加一
always@(posedge iic_clk or posedge iic_rst)begin
    if(iic_rst)
        bit_cnt <= 'd0;
    else if(iic_state == IDLE)
        bit_cnt <= 'd0;
    else if( (bit_cnt == bit_cnt_num - 1) && (cnt_scl == SCL_CNT_MAX) ) //当前状态下 最后一位传输完成 位计数器清零
        bit_cnt <= 'd0;
    else if(cnt_scl == SCL_CNT_MAX) //当分频计数器计数到最大值时 代表1位传输完成 位计数器加1
        bit_cnt <= bit_cnt + 1'b1;
    else
        bit_cnt <= bit_cnt;
end

//用于表示每个状态每次发送的数据位数, 发送器件地址之前需要发送起始位, 在加上应答位, 需要 10 个 SCL时钟。
//其余状态每次发送一字节数据后需要发送应答位, 所以计数器最大值为9。
always@(posedge iic_clk or posedge iic_rst)begin
    if(iic_rst)
        bit_cnt_num <= 'd9;
    else if((iic_state == START_DEVICE_ADDR) || (iic_state == R_START_DEVICE_ADDR)) //起始位 + 写器件地址 + 读/写位 + 应答位 = 10bit
        bit_cnt_num <= 'd10;
    else
        bit_cnt_num <= 'd9;    //8位 + 1位应答位 = 9bit
end

//*****字节计数器 & 更新每个状态下字节数最大值***** (用于解决 数据地址, 读写传输数据, 字节数可变的问题) *****/
//byte_cnt 字节计数器, 用于计数 数据传输中的 字节数
always@(posedge iic_clk or posedge iic_rst)begin
    if(iic_rst)
        byte_cnt <= 'd0;

```

```

else if(iic_state == IDLE)
    byte_cnt <= 'd0;
else if( end_byte_flag )
    byte_cnt <= 'd0;
else if( add_byte_flag)
    byte_cnt <= byte_cnt + 1'b1;
else
    byte_cnt <= byte_cnt;
end

//byte_cnt_num 传输字节计数器最大值
always@(posedge iic_clk or posedge iic_rst)begin
    if(iic_rst)
        byte_cnt_num <= 'd0 ;
    else if(iic_state == W_WORD_ADDR)
        byte_cnt_num <= P_ADDR_BYTE_NUM ;
    else if((iic_state == W_DATA) || (iic_state == R_DATA))
        byte_cnt_num <= P_DATA_BYTE_NUM ;
    else
        byte_cnt_num <= byte_cnt_num;
end

//*****IIC-SCL时钟的生成*****/
//iic_scl 生成串行时钟信号
always@(posedge iic_clk or posedge iic_rst)begin
    if(iic_rst)
        iic_scl <= 1'b1;
    else if(scl_pedge_flag || iic_state == IDLE)    //空闲状态 || 拉高条件有效 SCL=1
        iic_scl <= 1'b1;
    else if( ((iic_state == START_DEVICE_ADDR) && (bit_cnt > 0)) || (iic_state != START_DEVICE_ADDR) && scl_nedge_flag ) //发送起始位时不拉低 其余情况满足
        iic_scl <= 1'b0;
    else
        iic_scl <= iic_scl;
end

//*****IIC-SDA数据的切换与采集 *****/
//控制iic_SDA数据输出
always@(posedge iic_clk or posedge iic_rst)begin
    if(iic_rst)
        sda_out <= 1'b1;
    else begin
        case(iic_state)
            IDLE: sda_out <= 1'b1;
            START_DEVICE_ADDR:begin
                if( (bit_cnt < 9) && (w_sda_flag) ) //发送9位 起始位(1bit) + 写器件地址(7bit) + 写标志位(1bit) 发送完成后, 跳转到 写数据地址 状态 (应答主
                    sda_out <= start_device_write[8 - bit_cnt];
                else
                    sda_out <= sda_out;
            end
            W_WORD_ADDR:begin
                if( (bit_cnt < 8) && (w_sda_flag) ) //发送8位*N 需要写入的寄存器地址
                    sda_out <= word_addr_reg[P_ADDR_BYTE_NUM*8 -1 - (byte_cnt*8) - bit_cnt];
                else
                    sda_out <= sda_out;
            end
            W_DATA:begin
                if( (bit_cnt < 8) && (w_sda_flag) ) //发送8位*N 输出写入的数据 先输出高字节数据
                    sda_out <= wdata_reg[P_DATA_BYTE_NUM*8 -1 - (byte_cnt*8) - bit_cnt];
                else
                    sda_out <= sda_out;
            end
            R_START_DEVICE_ADDR:begin
                if( ((bit_cnt == 0) || (bit_cnt == bit_cnt_num - 2)) && (w_sda_flag) ) // (1. 第1位的起始位 便于后面高电平期间拉低 2. 倒数第2位的写数据位 便于
                    sda_out <= 1'b1;
                else if(w_sda_flag)
                    sda_out <= start_device_read[8 - bit_cnt]; //发送起始位之后 发送中间的7位器件地址位
                else if(r_sda_flag && (bit_cnt == 0)) //SCL的高电平期间 产生下降沿 再次发送起始位
                    sda_out <= 1'b0;
                else
                    sda_out <= sda_out;
            end
            R_DATA:begin
                if( (byte_cnt == P_DATA_BYTE_NUM - 1) && ((bit_cnt == bit_cnt_num - 1) && (w_sda_flag)) )
                    sda_out <= 1'b1;
                else if((bit_cnt == bit_cnt_num - 1) && (w_sda_flag))
                    sda_out <= 1'b0;
                else
                    sda_out <= sda_out;
            end
        endcase
    end
end

```

```

        STOP:begin
            if(w_sda_flag)          //在最后一个完整时钟周期内 停止信号 写数据期间 需要先拉低 (让后面SDA可以产生下降沿)
                sda_out <= 1'b0;
            else if(r_sda_flag)      //在最后一个完整时钟周期内 停止信号 读数据期间 需要拉高 (在SCL高电平期间 产生上升沿)
                sda_out <= 1'b1;
            else
                sda_out <= sda_out;
            end
            default:sda_out <= sda_out;
        endcase
    end
end

/*****SDA输出, 输入模式 切换*****/
//sda_ctrl 主机接收从机数据 && 主机对从机进行应答 sda_ctrl= 1'b0
always@(posedge iic_clk or posedge iic_rst)begin
    if(iic_rst)
        sda_ctrl <= 1'b1;
    else begin
        case(iic_state)
            START_DEVICE_ADDR, R_START_DEVICE_ADDR, W_WORD_ADDR, W_DATA:begin
                if((w_sda_flag) && (bit_cnt == 0)) //bit计数器为0时, 总线拉高, 开始写入下一字节数据, 主机控制SDA, 进行输出
                    sda_ctrl <= 1'b1;
                else if((w_sda_flag) && (bit_cnt == bit_cnt_num - 1)) //当写入最后一位数据时, 从机控制SDA, 进行应答输入
                    sda_ctrl <= 1'b0;
            end
            R_DATA:begin
                if((w_sda_flag) && (bit_cnt == 0)) //读取数据阶段, 开始接收数据时, 从机控制SDA, 进行数据输入
                    sda_ctrl <= 1'b0;
                else if((w_sda_flag) && (bit_cnt == bit_cnt_num - 1)) //读取数据阶段 主机控制SDA, 进行应答输出
                    sda_ctrl <= 1'b1;
            end
            STOP:begin
                if((w_sda_flag) && (bit_cnt == 0)) //停止状态, 主机控制SDA, 进行数据输出
                    sda_ctrl <= 1'b1;
                else
                    sda_ctrl <= sda_ctrl;
            end
            default:sda_ctrl <= sda_ctrl;
        endcase
    end
end

/*****主机接收从机数据*****/
//rdata_reg 对从机输入数据寄存
always@(posedge iic_clk or posedge iic_rst)begin
    if(iic_rst)
        rdata_reg <= 'd0;
    else if((iic_state == R_DATA) && (r_sda_flag) && (bit_cnt < 'd8))
        rdata_reg[P_DATA_BYTE_NUM*8 - 1 - (byte_cnt*8) - bit_cnt] <= sda_in;
    else
        rdata_reg <= rdata_reg;
end

//rdata_valid_reg 接收从机数据有效信号
always@(posedge iic_clk or posedge iic_rst)begin
    if(iic_rst)
        rdata_valid_reg <= 'd0;
    else if((iic_state == R_DATA) && (byte_cnt == P_DATA_BYTE_NUM - 1) && (r_sda_flag) && (bit_cnt == bit_cnt_num - 2))
        rdata_valid_reg <= 1'b1;
    else
        rdata_valid_reg <= 1'b0;
end

//将读取的数据取出
always@(posedge iic_clk or posedge iic_rst)begin
    if(iic_rst)begin
        iic_rdata <= 'd0;
        iic_rdata_valid <= 'd0;
    end
    else begin
        iic_rdata <= rdata_reg;
        iic_rdata_valid <= rdata_valid_reg;
    end
end

/*****应答失败指示信号*****/
//ack_error 从机应答错误信号, 输出高电平 用于指示从机应答有错误
always@(posedge iic_clk or posedge iic_rst)begin

```



```

if(iic_rst)
    iic_ack_error <= 1'b0;
else if(active)
    iic_ack_error <= 1'b0;
else if( ((iic_state == START_DEVICE_ADDR) ||
          (iic_state == W_WORD_ADDR) ||
          (iic_state == W_DATA) ||
          (iic_state == R_START_DEVICE_ADDR)) && (r_sda_flag) && (bit_cnt == bit_cnt_num - 1'b1)
        )
    iic_ack_error <= sda_in; //在从机应答时间段 将从机应答的信号 引出
else
    iic_ack_error <= iic_ack_error;
end

endmodule

```

1.9 key_ctrl.v

```

module key_ctrl (
    input wire      clk_100, // 输入时钟信号(100Hz)
    input wire      rst_n,   // 低电平复位信号
    input wire [3:0] key_in,  // 4 个按键输入信号
    output reg [3:0] down,    // 按键按下检测信号 (下降沿触发)
    output reg [3:0] up       // 按键释放检测信号 (上升沿触发)
);

reg [3:0] val; // 当前按键状态
reg [3:0] old; // 上一个时钟周期的按键状态

// 按键状态检测逻辑
always @(posedge clk_100 or negedge rst_n) begin
    if (!rst_n) begin
        // 复位时将所有状态设为未按下
        val <= 4'b1111; // 假设按键为低电平有效, 初始化为高电平未按下状态
        old <= 4'b1111; // 上一个状态初始化为高电平
        down <= 4'b0000; // 初始时没有按键按下
        up <= 4'b0000; // 初始时没有按键释放
    end else begin
        old <= val; // 保存当前状态到 old 以便下一个周期比较
        val <= key_in; // 更新 val 为当前的按键状态

        // 检测按键按下: val 和 old 比较, 下降沿检测
        down <= val & (val ^ old); // val 为低且 val 与 old 不同, 说明按键刚被按下

        // 检测按键释放: val 和 old 比较, 上升沿检测
        up <= ~val & (val ^ old); // val 为高且 val 与 old 不同, 说明按键刚被释放
    end
end

endmodule

```

1.10 led_display.v

```

module led_display (
    input wire [7:0] led_pattern,
    output reg [7:0] led
);

always @(*) begin
    led = ~led_pattern;
end

endmodule

```

1.11 seg_driver.v

```

module seg_driver (
    input wire [3:0] digit, // 输入的十进制数字 (0-9)
    input wire      rst_n,  // 复位信号 (低电平有效)
    input wire      current_dp, // 小数点使能信号
    input wire [2:0] position, // 当前显示的位
    output reg [7:0] digit_sel, // 位选信号
    output reg [7:0] segment_data // 段选信号
);

// 段码定义 (每个数字对应的七段显示编码)

```

```

localparam [7:0] SEG_0 = 8'b1100_0000; //0
localparam [7:0] SEG_1 = 8'b1111_1001; //1
localparam [7:0] SEG_2 = 8'b1010_0100; //2
localparam [7:0] SEG_3 = 8'b1011_0000; //3
localparam [7:0] SEG_4 = 8'b1001_1001; //4
localparam [7:0] SEG_5 = 8'b1001_0010; //5
localparam [7:0] SEG_6 = 8'b1000_0010; //6
localparam [7:0] SEG_7 = 8'b1111_1000; //7
localparam [7:0] SEG_8 = 8'b1000_0000; //8
localparam [7:0] SEG_9 = 8'b1001_0000; //9
localparam [7:0] SEG_C = 8'b1100_0110; //C
localparam [7:0] SEG_F = 8'b1000_1110; //F
localparam [7:0] SEG_DASH = 8'b1011_1111; //-
localparam [7:0] SEG_NULL = 8'b1111_1111; //灭

// 根据输入的数字和小数点状态生成段码
always @(*) begin
    if (!rst_n) begin
        segment_data = 8'b1111_1111; // 复位时熄灭所有段码
        digit_sel = 8'b1111_1111; // 复位时关闭所有位选
    end else begin
        // 根据输入的数字选择对应的段码
        case (digit)
            4'd0: segment_data = SEG_0;
            4'd1: segment_data = SEG_1;
            4'd2: segment_data = SEG_2;
            4'd3: segment_data = SEG_3;
            4'd4: segment_data = SEG_4;
            4'd5: segment_data = SEG_5;
            4'd6: segment_data = SEG_6;
            4'd7: segment_data = SEG_7;
            4'd8: segment_data = SEG_8;
            4'd9: segment_data = SEG_9;
            4'd10: segment_data = SEG_NULL; // 灭
            4'd11: segment_data = SEG_DASH; // -
            4'd12: segment_data = SEG_C; // C
            4'd15: segment_data = SEG_F; // F
            default: segment_data = SEG_NULL; // 无效输入时熄灭所有段码
        endcase

        // 如果 decimal_en 为 1, 则激活小数点 (最高位为小数点位)
        if (current_dp) segment_data[7] = 1'b0;

        // 根据当前位位置 `position` 控制位选信号
        digit_sel = ~(8'b0000_0001 << position); // 按位激活
    end
end

endmodule

```

1.12 spi_master.v

```

module spi_master
#(
    parameter          SYS_CLK          = 28'd50_000_000 , //系统时钟频率
    parameter          SPI_SCLK         = 28'd100_000    , //SPI_SCLK频率
    parameter          SPI_CPOL         = 1'b0           , //时钟极性
    parameter          SPI_CPHA         = 1'b0           , //时钟相位
)
(
    input               spi_clk          ,
    input               spi_rst          ,

    input wire          spi_cs_ctrl      , //片选控制端口
    input wire          spi_wr_en        , //传输使能
    input wire          [ 7: 0] spi_data_in , //数据输入
    output wire         [ 7: 0] spi_data_out , //数据输出
    output wire         spi_wr_ack       , //传输结束应答

    //SPI物理端口
    output wire         ds1302_ce        , //片选端口
    output reg          ds1302_sclk      , //SPI时钟
    output wire         spi_mosi         , //用三态门进行构建
    input wire          spi_miso

);

localparam            IDLE              = 0              ; //空闲状态

```

```

localparam          H_SCLK_IDLE      = 1          ; //SCLK前半周期
localparam          SCLK_EDGE        = 2          ; //SCLK边沿时刻
localparam          L_HALF_CYCLE     = 3          ; //SCLK后半周期
localparam          ACK              = 4          ; //应答
localparam          ACK_WAIT         = 5          ;

reg                  [ 3: 0]          spi_state    ;
reg                  [ 27: 0]         spi_sclk_cnt ;
reg                  [ 27: 0]         spi_edge_cnt ;

reg                  [ 7: 0]          mosi_shift   ;
reg                  [ 7: 0]          miso_shift   ;

assign               spi_mosi         = mosi_shift[7] ;
assign               spi_data_out     = miso_shift   ;
assign               ds1302_ce       = spi_cs_ctrl   ;
assign               spi_wr_ack      = (spi_state == ACK);

always@(posedge spi_clk or posedge spi_rst) begin
    if(spi_rst)
        spi_state <= IDLE;
    else begin
        case(spi_state)
            IDLE :
                if(spi_wr_en)
                    spi_state <= H_SCLK_IDLE;
                else
                    spi_state <= IDLE;
            H_SCLK_IDLE :
                if(spi_sclk_cnt == 'd50)
                    spi_state <= SCLK_EDGE;
                else
                    spi_state <= H_SCLK_IDLE;
            SCLK_EDGE :
                if(spi_edge_cnt == 'd15)
                    spi_state <= L_HALF_CYCLE;
                else
                    spi_state <= H_SCLK_IDLE;
            L_HALF_CYCLE:
                if(spi_sclk_cnt == 'd50)
                    spi_state <= ACK;
                else
                    spi_state <= L_HALF_CYCLE;
            ACK :spi_state <= ACK_WAIT;
            ACK_WAIT :spi_state <= IDLE;
            default:spi_state <= IDLE;
        endcase
    end
end

//ds1302_sclk SPI_SLCK时钟翻转
always@(posedge spi_clk or posedge spi_rst) begin
    if(spi_rst)
        ds1302_sclk <= 'd0;
    else if(spi_state == IDLE)
        ds1302_sclk <= SPI_CPOL;
    else if(spi_state == SCLK_EDGE)
        ds1302_sclk <= ~ds1302_sclk;
    else
        ds1302_sclk <= ds1302_sclk;
end

//spi_sclk_cnt: SPI_SLCK产生分频计数器
always@(posedge spi_clk or posedge spi_rst) begin
    if(spi_rst)
        spi_sclk_cnt <= 'd0;
    else if(spi_state == IDLE)
        spi_sclk_cnt <= 'd0;
    else if(spi_state == H_SCLK_IDLE || spi_state == L_HALF_CYCLE)
        spi_sclk_cnt <= spi_sclk_cnt + 1'b1;
    else
        spi_sclk_cnt <= 'd0;
end

//spi_edge_cnt:SPI_SCLK边沿计数器
always@(posedge spi_clk or posedge spi_rst) begin
    if(spi_rst)
        spi_edge_cnt <= 'd0;
    else if(spi_state == IDLE)

```

```

        spi_edge_cnt <= 'd0;
    else if(spi_state == SCLK_EDGE)
        spi_edge_cnt <= spi_edge_cnt + 1'b1;
    else
        spi_edge_cnt <= spi_edge_cnt;
end

//MOSI
always@(posedge spi_clk or posedge spi_rst) begin
    if(spi_rst)
        mosi_shift <= 'd0;
    else if(spi_state == IDLE && spi_wr_en)
        mosi_shift <= spi_data_in;
    else if((spi_state == SCLK_EDGE) && (SPI_CPHA == 1'b0) && (spi_edge_cnt[0] == 1'b1))
        mosi_shift <= {mosi_shift[6:0],mosi_shift[7]};
    else if((spi_state == SCLK_EDGE) && (SPI_CPHA == 1'b1) && (spi_edge_cnt[0] == 1'b0) && (spi_edge_cnt != 'd0))
        mosi_shift <= {mosi_shift[6:0],mosi_shift[7]};
end

//MISO
always@(posedge spi_clk or posedge spi_rst) begin
    if(spi_rst)
        miso_shift <= 'd0;
    else if(spi_state == IDLE && spi_wr_en)
        miso_shift <= 'd0;
    else if((spi_state == SCLK_EDGE) && (spi_edge_cnt[0] == 1'b0) && (SPI_CPHA == 1'b0))
        miso_shift <= {miso_shift[6:0],spi_miso};
    else if((spi_state == SCLK_EDGE) && (spi_edge_cnt[0] == 1'b1) && (SPI_CPHA == 1'b1))
        miso_shift <= {miso_shift[6:0],spi_miso};
end

endmodule

```

1.13 sram_ctrl.v

```

module sram_ctrl #(
    parameter READ_WAIT_CYCLES = 2,
    parameter WRITE_PULSE_CYCLES = 2
) (
    input wire      clk,
    input wire      rst_n,
    input wire      read_req,
    input wire      write_req,
    input wire [16:0] addr,
    input wire [ 7:0] write_data,
    output reg [ 7:0] read_data,
    output wire      ready,
    output reg       done,
    output wire [16:0] sram_addr,
    inout wire [ 7:0] sram_data,
    output wire      sram_ce_n,
    output wire      sram_oe_n,
    output wire      sram_we_n
);

    localparam [1:0] S_IDLE  = 2'd0;
    localparam [1:0] S_READ  = 2'd1;
    localparam [1:0] S_WRITE = 2'd2;
    localparam [1:0] S_DONE  = 2'd3;

    reg [1:0] state;
    reg [16:0] addr_reg;
    reg [ 7:0] write_data_reg;
    reg [ 7:0] wait_cnt;

    wire [7:0] sram_data_in;
    reg [7:0] sram_data_out;
    reg       sram_data_oe;

    assign ready      = (state == S_IDLE);
    assign sram_addr  = addr_reg;
    assign sram_data  = sram_data_oe ? sram_data_out : 8'bz;
    assign sram_data_in = sram_data;

    assign sram_ce_n = (state == S_READ || state == S_WRITE) ? 1'b0 : 1'b1;
    assign sram_oe_n = (state == S_READ) ? 1'b0 : 1'b1;
    assign sram_we_n = (state == S_WRITE) ? 1'b0 : 1'b1;

```

```

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state          <= S_IDLE;
        addr_reg        <= 17'd0;
        write_data_reg  <= 8'd0;
        read_data       <= 8'd0;
        wait_cnt        <= 8'd0;
        done            <= 1'b0;
        sram_data_out    <= 8'd0;
        sram_data_oe     <= 1'b0;
    end else begin
        done <= 1'b0;

        case (state)
            S_IDLE: begin
                wait_cnt    <= 8'd0;
                sram_data_oe <= 1'b0;

                if (write_req) begin
                    addr_reg    <= addr;
                    write_data_reg <= write_data;
                    sram_data_out <= write_data;
                    sram_data_oe <= 1'b1;
                    state        <= S_WRITE;
                end else if (read_req) begin
                    addr_reg    <= addr;
                    sram_data_oe <= 1'b0;
                    state        <= S_READ;
                end
            end

            S_READ: begin
                if (wait_cnt == READ_WAIT_CYCLES - 1) begin
                    read_data <= sram_data_in;
                    wait_cnt  <= 8'd0;
                    state     <= S_DONE;
                end else begin
                    wait_cnt <= wait_cnt + 1'b1;
                end
            end

            S_WRITE: begin
                sram_data_oe <= 1'b1;
                sram_data_out <= write_data_reg;

                if (wait_cnt == WRITE_PULSE_CYCLES - 1) begin
                    wait_cnt <= 8'd0;
                    state     <= S_DONE;
                end else begin
                    wait_cnt <= wait_cnt + 1'b1;
                end
            end

            S_DONE: begin
                sram_data_oe <= 1'b0;
                done         <= 1'b1;
                state        <= S_IDLE;
            end

            default: begin
                state <= S_IDLE;
            end
        endcase
    end
end
endmodule

```

1.14 uart_rx.v

/* UART接收模块

一. 模块功能:

1. 支持可配置的波特率
2. 支持8位数据位, 1位起始位, 1位停止位
3. 支持无校验位
4. 支持数据接收完成标志输出
5. 支持数据同步和亚稳态处理

二. 模块说明:

1. 使用状态机实现UART接收时序
2. 波特率通过参数配置, 默认115200
3. 系统时钟频率通过参数配置, 默认50MHz
4. 使用两级寄存器同步输入数据, 消除亚稳态

*/

```

module uart_rx #(
    parameter UART_BPS = 115200,    // 串口波特率 (默认115200)
    parameter CLK_FREQ = 50_000_000 // 系统时钟频率 (默认50MHz)
) (
    input wire      clk,           // 输入: 系统时钟信号 (50MHz)
    input wire      rst_n,        // 输入: 全局复位信号 (低电平有效)
    input wire      rx,           // 输入: 串行数据输入
    output reg [7:0] po_data,      // 输出: 8位并行数据
    output reg      po_flag       // 输出: 数据有效标志信号 (高电平有效)
);

// 定义参数和内部信号
localparam BAUD_CNT_MAX = CLK_FREQ / UART_BPS; // 计算波特率计数器最大值

// 定义状态机的状态 (独热码)
localparam IDLE = 3'b000; // 空闲状态: 等待起始位
localparam START = 3'b001; // 起始位检测状态: 检测起始位下降沿
localparam RECEIVE = 3'b010; // 数据接收状态: 接收8位数据
localparam STOP = 3'b011; // 停止位检测状态: 检测停止位

// 定义寄存器
reg [2:0] state; // 状态寄存器: 当前状态
reg [2:0] next_state; // 状态寄存器: 下一状态
reg [12:0] baud_cnt; // 波特率计数器: 用于生成正确的波特率
reg [3:0] bit_cnt; // 位计数器: 用于计数已接收的数据位
reg [7:0] rx_data; // 数据寄存器: 暂存接收的数据
reg rx_reg1, rx_reg2; // 同步寄存器: 用于消除亚稳态

// 数据同步, 消除亚稳态
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        rx_reg1 <= 1'b1; // 复位时置高电平
        rx_reg2 <= 1'b1; // 复位时置高电平
    end else begin
        rx_reg1 <= rx; // 第一级同步: 采样输入数据
        rx_reg2 <= rx_reg1; // 第二级同步: 进一步稳定数据
    end
end

// 波特率计数器逻辑
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) baud_cnt <= 13'b0; // 复位时清零
    else if (state == IDLE || baud_cnt == BAUD_CNT_MAX - 1)
        baud_cnt <= 13'b0; // 空闲状态或计数到最大值时清零
    else baud_cnt <= baud_cnt + 1'b1; // 正常计数
end

// 状态机: 状态转移逻辑
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) state <= IDLE; // 复位时进入空闲状态
    else state <= next_state; // 正常工作时状态转移
end

// 状态机: 下一状态逻辑
always @(*) begin
    case (state)
        IDLE: begin
            if (!rx_reg2) // 检测到起始位的下降沿
                next_state = START; // 进入起始位检测状态
            else next_state = IDLE; // 保持空闲状态
        end
        START: begin
            if (baud_cnt == BAUD_CNT_MAX / 2 - 1) // 到达起始位采样中点
                next_state = RECEIVE; // 进入数据接收状态
            else next_state = START; // 保持起始位检测状态
        end
        RECEIVE: begin
            if (bit_cnt == 4'd8 && baud_cnt == BAUD_CNT_MAX - 1) // 8位数据接收完成
                next_state = STOP; // 进入停止位检测状态
            else next_state = RECEIVE; // 保持数据接收状态
        end
        STOP: begin
    
```

```

        if (baud_cnt == BAUD_CNT_MAX - 1) // 停止位采样完成
            next_state = IDLE; // 进入空闲状态
        else next_state = STOP; // 保持停止位检测状态
    end
    default: next_state = IDLE; // 默认进入空闲状态
endcase
end

// 位计数器逻辑
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) bit_cnt <= 4'b0; // 复位时清零
    else if (state == RECEIVE && baud_cnt == BAUD_CNT_MAX - 1)
        bit_cnt <= bit_cnt + 1'b1; // 数据位接收完成时加1
    else if (state != RECEIVE) bit_cnt <= 4'b0; // 非数据接收状态时清零
end

// 数据接收逻辑
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) rx_data <= 8'b0; // 复位时清零
    else if (state == RECEIVE && baud_cnt == BAUD_CNT_MAX / 2 - 1) // 在数据位中点采样
        rx_data <= {rx_reg2, rx_data[7:1]}; // 右移并保存接收到的位
end

// 数据输出逻辑
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        po_data <= 8'b0; // 复位时输出数据清零
        po_flag <= 1'b0; // 复位时标志信号清零
    end else if (state == STOP && baud_cnt == BAUD_CNT_MAX - 1) begin
        po_data <= rx_data; // 停止位接收完成时输出数据
        po_flag <= 1'b1; // 停止位接收完成时标志信号置高
    end else po_flag <= 1'b0; // 其他状态标志信号清零
end

endmodule

```

1.15 uart_tx.v

```

/* UART发送模块
一. 模块功能：
1. 支持可配置的波特率
2. 支持8位数据位，1位起始位，1位停止位
3. 支持无校验位
4. 支持数据发送完成标志输出

二. 模块说明：
1. 使用状态机实现UART发送时序
2. 波特率通过参数配置，默认115200
3. 系统时钟频率通过参数配置，默认50MHz
*/

module uart_tx #(
    parameter UART_BPS = 115200, // 串口波特率（默认115200）
    parameter CLK_FREQ = 50_000_000 // 系统时钟频率（默认50MHz）
) (
    input wire      clk,        // 输入：系统时钟信号（50MHz）
    input wire      rst_n,      // 输入：全局复位信号（低电平有效）
    input wire [7:0] pi_data,    // 输入：8位并行数据
    input wire      pi_flag,    // 输入：并行数据有效标志信号（高电平有效）
    output reg      tx,         // 输出：串行数据输出
    output reg      tx_done     // 输出：数据发送完成标志（高电平有效）
);

// 定义参数和内部信号
localparam BAUD_CNT_MAX = CLK_FREQ / UART_BPS; // 计算波特率计数器最大值

// 定义状态机的状态（独热码）
localparam IDLE = 3'b000; // 空闲状态：等待数据发送请求
localparam START = 3'b001; // 起始位状态：发送起始位（低电平）
localparam DATA = 3'b010; // 数据位状态：发送8位数据
localparam STOP = 3'b011; // 停止位状态：发送停止位（高电平）

reg [2:0] state, next_state; // 状态寄存器：当前状态和下一状态
reg [12:0] baud_cnt; // 波特率计数器：用于生成正确的波特率
reg [3:0] bit_cnt; // 数据位计数器：用于计数已发送的数据位

// 状态机：状态转移逻辑

```

```

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) state <= IDLE; // 复位时进入空闲状态
    else state <= next_state; // 正常工作时状态转移
end

// 状态机：下一状态逻辑
always @(*) begin
    case (state)
        IDLE:
            if (pi_flag) next_state = START; // 收到数据有效标志，进入起始位状态
            else next_state = IDLE; // 无数据发送请求，保持空闲状态
        START:
            if (baud_cnt == BAUD_CNT_MAX - 1)
                next_state = DATA; // 起始位发送完成，进入数据位状态
            else next_state = START; // 起始位未发送完成，保持起始位状态
        DATA:
            if ((bit_cnt == 4'd7) && (baud_cnt == BAUD_CNT_MAX - 1))
                next_state = STOP; // 所有数据位发送完成，进入停止位状态
            else next_state = DATA; // 数据位未发送完成，保持数据位状态
        STOP:
            if (baud_cnt == BAUD_CNT_MAX - 1)
                next_state = IDLE; // 停止位发送完成，进入空闲状态
            else next_state = STOP; // 停止位未发送完成，保持停止位状态
        default: next_state = IDLE; // 默认状态为空闲状态
    endcase
end

// 波特率计数器逻辑
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) baud_cnt <= 13'b0; // 复位时清零
    else if (state == IDLE || baud_cnt == BAUD_CNT_MAX - 1)
        baud_cnt <= 13'b0; // 空闲状态或计数到最大值时清零
    else baud_cnt <= baud_cnt + 1'b1; // 正常计数
end

// 数据位计数器逻辑
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) bit_cnt <= 4'b0; // 复位时清零
    else if (state == DATA && baud_cnt == BAUD_CNT_MAX - 1)
        bit_cnt <= bit_cnt + 1'b1; // 数据位状态且波特率计数到最大值时加1
    else if (state != DATA) bit_cnt <= 4'b0; // 非数据位状态时清零
end

// 串行输出数据逻辑
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) tx <= 1'b1; // 复位时输出高电平
    else begin
        case (state)
            IDLE: tx <= 1'b1; // 空闲状态输出高电平
            START: tx <= 1'b0; // 起始位输出低电平
            DATA: tx <= pi_data[bit_cnt]; // 数据位按位输出
            STOP: tx <= 1'b1; // 停止位输出高电平
            default: tx <= 1'b1; // 默认输出高电平
        endcase
    end
end

// 数据发送完成标志逻辑
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) tx_done <= 1'b0; // 复位时清零
    else if (state == STOP && baud_cnt == BAUD_CNT_MAX - 1)
        tx_done <= 1'b1; // 停止位发送完成，标志置高
    else if (state == IDLE) tx_done <= 1'b0; // 空闲状态时清零
end

endmodule

```

1.16 w25q128_ctrl.v

```

module w25q128_ctrl #(
    parameter P_SYS_CLK = 50_000_000,
    parameter P_SPI_CLK = 2_000_000
) (
    input wire      clk,
    input wire      rst_n,
    input wire      start,
    input wire [2:0] op_code,
    input wire [23:0] flash_addr,

```



```

input wire [ 7:0] write_data,
output reg [23:0] jedec_id,
output reg [ 7:0] read_data,
output reg [ 7:0] status_reg,
output wire      ready,
output wire      busy,
output reg       done,
output wire      flash_cs_n,
output wire      flash_sck,
inout wire       flash_io0,
inout wire       flash_io1,
inout wire       flash_io2,
inout wire       flash_io3
);

localparam [2:0] OP_READ_JEDEC_ID = 3'd0;
localparam [2:0] OP_READ_STATUS  = 3'd1;
localparam [2:0] OP_READ_BYTE    = 3'd2;
localparam [2:0] OP_WRITE_BYTE   = 3'd3;
localparam [2:0] OP_SECTOR_ERASE = 3'd4;

localparam [2:0] S_IDLE          = 3'd0;
localparam [2:0] S_LAUNCH_WREN  = 3'd1;
localparam [2:0] S_WAIT_WREN    = 3'd2;
localparam [2:0] S_LAUNCH_MAIN  = 3'd3;
localparam [2:0] S_WAIT_MAIN    = 3'd4;
localparam [2:0] S_LAUNCH_POLL  = 3'd5;
localparam [2:0] S_WAIT_POLL    = 3'd6;
localparam [2:0] S_DONE         = 3'd7;

localparam [1:0] SPI_IDLE       = 2'd0;
localparam [1:0] SPI_TRANSFER  = 2'd1;
localparam [1:0] SPI_FINISH    = 2'd2;

localparam integer SPI_DIV = ((P_SYS_CLK / (P_SPI_CLK * 2)) < 1) ? 1 : (P_SYS_CLK / (P_SPI_CLK * 2));

reg [2:0] state;
reg [2:0] op_code_reg;
reg [23:0] flash_addr_reg;
reg [7:0] write_data_reg;

reg      spi_start;
reg [39:0] spi_tx_word;
reg [ 5:0] spi_total_bits;
reg [ 5:0] spi_capture_bits;

reg [ 1:0] spi_state;
reg [39:0] spi_shift_reg;
reg [31:0] spi_rx_shift;
reg [31:0] spi_rx_data;
reg      spi_done;
reg [ 5:0] spi_bit_index;
reg [ 5:0] spi_rx_start;
reg [ 5:0] spi_total_bits_reg;
reg [15:0] spi_div_cnt;
reg      flash_cs_n_reg;
reg      flash_sck_reg;
reg      flash_io0_out;

wire flash_miso;

assign ready = (state == S_IDLE);
assign busy  = (state != S_IDLE);

assign flash_cs_n = flash_cs_n_reg;
assign flash_sck  = flash_sck_reg;

assign flash_io0 = flash_io0_out;
assign flash_io1 = 1'bz;
assign flash_io2 = 1'bz;
assign flash_io3 = 1'bz;
assign flash_miso = flash_io1;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state          <= S_IDLE;
        op_code_reg     <= OP_READ_JEDEC_ID;
        flash_addr_reg  <= 24'd0;
        write_data_reg  <= 8'd0;
    end
end

```

```

jedec_id      <= 24'd0;
read_data     <= 8'd0;
status_reg    <= 8'd0;
done          <= 1'b0;
spi_start     <= 1'b0;
spi_tx_word   <= 40'd0;
spi_total_bits <= 6'd0;
spi_capture_bits <= 6'd0;
end else begin
done          <= 1'b0;
spi_start     <= 1'b0;

case (state)
  S_IDLE: begin
    if (start) begin
      op_code_reg    <= op_code;
      flash_addr_reg <= flash_addr;
      write_data_reg <= write_data;

      if (op_code == OP_WRITE_BYTE || op_code == OP_SECTOR_ERASE)
        state <= S_LAUNCH_WREN;
      else
        state <= S_LAUNCH_MAIN;
    end
  end
end

S_LAUNCH_WREN: begin
  spi_tx_word    <= {8'h06, 32'h0000_0000};
  spi_total_bits <= 6'd8;
  spi_capture_bits <= 6'd0;
  spi_start      <= 1'b1;
  state          <= S_WAIT_WREN;
end

S_WAIT_WREN: begin
  if (spi_done)
    state <= S_LAUNCH_MAIN;
end

S_LAUNCH_MAIN: begin
  case (op_code_reg)
    OP_READ_JEDEC_ID: begin
      spi_tx_word    <= {8'h9F, 24'h00_0000, 8'h00};
      spi_total_bits <= 6'd32;
      spi_capture_bits <= 6'd24;
    end

    OP_READ_STATUS: begin
      spi_tx_word    <= {8'h05, 8'h00, 24'h00_0000};
      spi_total_bits <= 6'd16;
      spi_capture_bits <= 6'd8;
    end

    OP_READ_BYTE: begin
      spi_tx_word    <= {8'h03, flash_addr_reg, 8'h00};
      spi_total_bits <= 6'd40;
      spi_capture_bits <= 6'd8;
    end

    OP_WRITE_BYTE: begin
      spi_tx_word    <= {8'h02, flash_addr_reg, write_data_reg};
      spi_total_bits <= 6'd40;
      spi_capture_bits <= 6'd0;
    end

    OP_SECTOR_ERASE: begin
      spi_tx_word    <= {8'h20, flash_addr_reg, 8'h00};
      spi_total_bits <= 6'd32;
      spi_capture_bits <= 6'd0;
    end

    default: begin
      spi_tx_word    <= {8'h05, 8'h00, 24'h00_0000};
      spi_total_bits <= 6'd16;
      spi_capture_bits <= 6'd8;
    end
  endcase

  spi_start <= 1'b1;

```

```

        state    <= S_WAIT_MAIN;
    end

    S_WAIT_MAIN: begin
        if (spi_done) begin
            case (op_code_reg)
                OP_READ_JEDEC_ID: begin
                    jedec_id <= spi_rx_data[23:0];
                    state    <= S_DONE;
                end

                OP_READ_STATUS: begin
                    status_reg <= spi_rx_data[7:0];
                    state    <= S_DONE;
                end

                OP_READ_BYTE: begin
                    read_data <= spi_rx_data[7:0];
                    state    <= S_DONE;
                end

                OP_WRITE_BYTE,
                OP_SECTOR_ERASE: begin
                    state <= S_LAUNCH_POLL;
                end

                default: begin
                    state <= S_DONE;
                end
            endcase
        end
    end

    S_LAUNCH_POLL: begin
        spi_tx_word    <= {8'h05, 8'h00, 24'h00_0000};
        spi_total_bits <= 6'd16;
        spi_capture_bits <= 6'd8;
        spi_start      <= 1'b1;
        state          <= S_WAIT_POLL;
    end

    S_WAIT_POLL: begin
        if (spi_done) begin
            status_reg <= spi_rx_data[7:0];

            if (spi_rx_data[0])
                state <= S_LAUNCH_POLL;
            else
                state <= S_DONE;
            end
        end
    end

    S_DONE: begin
        done <= 1'b1;
        state <= S_IDLE;
    end

    default: begin
        state <= S_IDLE;
    end
endcase
end
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        spi_state    <= SPI_IDLE;
        spi_shift_reg <= 40'd0;
        spi_rx_shift  <= 32'd0;
        spi_rx_data   <= 32'd0;
        spi_done      <= 1'b0;
        spi_bit_index <= 6'd0;
        spi_rx_start  <= 6'd0;
        spi_total_bits_reg <= 6'd0;
        spi_div_cnt   <= 16'd0;
        flash_cs_n_reg <= 1'b1;
        flash_sck_reg <= 1'b0;
        flash_io0_out <= 1'b0;
    end else begin

```

```

spi_done <= 1'b0;

case (spi_state)
  SPI_IDLE: begin
    flash_cs_n_reg <= 1'b1;
    flash_sck_reg  <= 1'b0;
    spi_div_cnt    <= 16'd0;
    spi_bit_index  <= 6'd0;

    if (spi_start) begin
      flash_cs_n_reg <= 1'b0;
      flash_sck_reg  <= 1'b0;
      spi_shift_reg  <= spi_tx_word;
      spi_rx_shift   <= 32'd0;
      spi_rx_start   <= spi_total_bits - spi_capture_bits;
      spi_total_bits_reg <= spi_total_bits;
      flash_io0_out  <= spi_tx_word[39];
      spi_state      <= SPI_TRANSFER;
    end
  end

  SPI_TRANSFER: begin
    if (spi_div_cnt == SPI_DIV - 1) begin
      spi_div_cnt <= 16'd0;

      if (!flash_sck_reg) begin
        flash_sck_reg <= 1'b1;

        if (spi_bit_index >= spi_rx_start && spi_capture_bits != 0)
          spi_rx_shift <= {spi_rx_shift[30:0], flash_miso};
        else begin
          flash_sck_reg <= 1'b0;

          if (spi_bit_index == spi_total_bits_reg - 1) begin
            spi_rx_data <= spi_rx_shift;
            spi_state   <= SPI_FINISH;
          end else begin
            spi_bit_index <= spi_bit_index + 1'b1;
            spi_shift_reg <= {spi_shift_reg[38:0], 1'b0};
            flash_io0_out <= spi_shift_reg[38];
          end
        end
      end else begin
        spi_div_cnt <= spi_div_cnt + 1'b1;
      end
    end
  end

  SPI_FINISH: begin
    flash_cs_n_reg <= 1'b1;
    flash_sck_reg  <= 1'b0;
    spi_done       <= 1'b1;
    spi_state      <= SPI_IDLE;
  end

  default: begin
    spi_state <= SPI_IDLE;
  end
endcase
end
endmodule

```

2. User 模块代码

2.1 ds1302_test.v

```

module ds1302_test (
    input wire rst,    // 复位信号, 高电平有效
    input wire clk,    // 时钟信号, 用于同步逻辑操作

    output wire ds1302_ce,    // DS1302 芯片使能信号
    output wire ds1302_sclk,  // DS1302 串行时钟信号
    inout  wire ds1302_io,    // DS1302 数据线 (双向)

    input wire [7:0] write_second,  // 写入的秒值
    input wire [7:0] write_minute,  // 写入的分值
    input wire [7:0] write_hour,    // 写入的小时值
    input wire [7:0] write_date,    // 写入的日期值
    input wire [7:0] write_month,   // 写入的月份值
    input wire [7:0] write_week,    // 写入的星期值
    input wire [7:0] write_year,    // 写入的年份值

    input wire      write_time_req, // 写入时间请求信号
    output wire [7:0] read_second,  // 读取到的秒值
    output wire [7:0] read_minute,  // 读取到的分值
    output wire [7:0] read_hour,    // 读取到的小时值
    output wire [7:0] read_date,    // 读取到的日期值
    output wire [7:0] read_month,   // 读取到的月份值
    output wire [7:0] read_week,    // 读取到的星期值
    output wire [7:0] read_year,    // 读取到的年份值
);

// 状态机变量定义
reg [2:0] state, next_state; // 当前状态和下一状态
reg      write_time_flag;    // 写时间触发信号
reg      clear_ch_flag;     // 清除 CH 位的触发信号
reg [7:0] write_second_reg;  // 写入秒值寄存器
reg [7:0] write_minute_reg;  // 写入分钟值寄存器
reg [7:0] write_hour_reg;    // 写入小时值寄存器
reg [7:0] write_date_reg;    // 写入日期值寄存器
reg [7:0] write_month_reg;   // 写入月份值寄存器
reg [7:0] write_week_reg;    // 写入星期值寄存器
reg [7:0] write_year_reg;    // 写入年份值寄存器
wire      write_time_ack;    // 写时间完成信号
wire      read_time_ack;     // 读时间完成信号
wire      CH;               // 秒值的最高位, 用于判断是否暂停

// 状态定义
localparam S_IDLE = 0; // 空闲状态
localparam S_READ_CH = 1; // 读取 CH 状态
localparam S_CLEAR_CH = 2; // 清除 CH 状态
localparam S_WRITE = 3; // 写时间状态
localparam S_WAIT = 4; // 等待状态
localparam S_READ_TIME = 5; // 读取时间状态

// CH 位的分配
assign CH = read_second[7];

// 写时间触发逻辑
always @(posedge clk or posedge rst) begin
    if (rst) begin
        write_time_flag <= 1'b0; // 复位时清除写时间标志
    end else if (write_time_req) begin
        write_time_flag <= 1'b1; // 收到写时间请求时设置标志
    end else if (write_time_ack) begin
        write_time_flag <= 1'b0; // 写完成后清除写时间标志
    end
end

// 清除 CH 触发逻辑
always @(posedge clk or posedge rst) begin
    if (rst) begin
        clear_ch_flag <= 1'b0; // 复位时清除清除 CH 标志
    end else if (state == S_CLEAR_CH && write_time_ack) begin
        clear_ch_flag <= 1'b0; // 清除 CH 完成后清零标志
    end else if (state == S_READ_CH && read_time_ack && CH) begin
        clear_ch_flag <= 1'b1; // 如果 CH 位为 1, 则触发清除
    end
end

```

```

    end
end

// 写入时间寄存器赋值逻辑
always @(posedge clk or posedge rst) begin
    if (rst) begin
        write_second_reg <= 8'h00; // 复位时清零
        write_minute_reg <= 8'h00;
        write_hour_reg   <= 8'h00;
        write_date_reg   <= 8'h00;
        write_month_reg  <= 8'h00;
        write_week_reg   <= 8'h00;
        write_year_reg   <= 8'h00;
    end else if (write_time_req) begin
        write_second_reg <= write_second; // 写入预设的时间数据
        write_minute_reg <= write_minute;
        write_hour_reg   <= write_hour;
        write_date_reg   <= write_date;
        write_month_reg  <= write_month;
        write_week_reg   <= write_week;
        write_year_reg   <= write_year;
    end else if (clear_ch_flag) begin
        write_second_reg <= {1'b0, read_second[6:0]}; // 清除 CH 位
    end
end

// 状态机控制逻辑
always @(posedge clk or posedge rst) begin
    if (rst) state <= S_IDLE; // 复位时进入空闲状态
    else state <= next_state; // 否则进入下一状态
end

// 状态机状态转移逻辑
always @(*) begin
    case (state)
        S_IDLE: begin
            if (write_time_flag) next_state <= S_WRITE; // 写时间触发时进入写状态
            else next_state <= S_READ_CH; // 否则进入读 CH 状态
        end
        S_READ_CH: begin
            if (read_time_ack)
                next_state <= (CH ? S_CLEAR_CH : S_READ_TIME); // CH 为 1 进入清除 CH, 否则直接读时间
            else next_state <= S_READ_CH;
        end
        S_CLEAR_CH: begin
            if (write_time_ack) next_state <= S_WAIT; // 清除 CH 完成后进入等待状态
            else next_state <= S_CLEAR_CH;
        end
        S_WRITE: begin
            if (write_time_ack) next_state <= S_WAIT; // 写完成后进入等待状态
            else next_state <= S_WRITE;
        end
        S_WAIT: begin
            next_state <= S_READ_TIME; // 等待完成后进入读时间状态
        end
        S_READ_TIME: begin
            if (read_time_ack) next_state <= S_IDLE; // 读完成后返回空闲状态
            else next_state <= S_READ_TIME;
        end
        default: next_state <= S_IDLE; // 默认返回空闲状态
    endcase
end

// 实例化 DS1302 控制模块
ds1302_wr_drive ds1302_ctrl_m0 (
    .ds1302_clk(clk), // 时钟信号
    .ds1302_rst(rst), // 复位信号
    .write_second(write_second_reg), // 写入的秒值
    .write_minute(write_minute_reg), // 写入的分钟值
    .write_hour(write_hour_reg), // 写入的小时值
    .write_date(write_date_reg), // 写入的日期值
    .write_month(write_month_reg), // 写入的月份值
    .write_week(write_week_reg), // 写入的星期值
    .write_year(write_year_reg), // 写入的年份值
    .write_time_req(clear_ch_flag || write_time_flag), // 写时间或清除 CH 触发信号
    .write_time_ack(write_time_ack), // 写时间完成信号
    .read_second(read_second), // 读取的秒值
    .read_minute(read_minute), // 读取的分钟值
    .read_hour(read_hour), // 读取的小时值

```

```

        .read_date(read_date), // 读取的日期值
        .read_month(read_month), // 读取的月份值
        .read_week(read_week), // 读取的星期值
        .read_year(read_year), // 读取的年份值
        .read_time_req(state == S_READ_CH || state == S_READ_TIME), // 读时间请求信号
        .read_time_ack(read_time_ack), // 读时间完成信号
        .ds1302_ce(ds1302_ce), // DS1302 芯片使能信号
        .ds1302_sclk(ds1302_sclk), // DS1302 串行时钟信号
        .ds1302_data(ds1302_io) // DS1302 数据线
    );

endmodule

```

2.2 key_proc.v

```

module key_proc (
    input wire      clk,          // 系统时钟信号
    input wire      rst_n,       // 低电平复位信号
    input wire [3:0] key_in,     // 4 个按键输入信号
    output reg [1:0] mode,       // 模式输出，用于切换 LED 显示模式
    output reg      write_pulse  // 写入数据的单周期脉冲信号
);

wire [3:0] down; // 每个按键的按下检测信号
wire [3:0] up;  // 每个按键的松开检测信号
wire      clk_100; // 100Hz 时钟，用于按键消抖

// 频率分频器生成 100Hz 时钟
frequency_divider u_frequency_divider (
    .clk      (clk),
    .rst_n    (rst_n),
    .clkbases(50_000_000),
    .clkdiv   (100),
    .clkout   (clk_100)
);

// 按键消抖模块
key_ctrl key_ctrl_inst (
    .clk_100(clk_100), // 系统时钟输入
    .rst_n   (rst_n),  // 低电平复位信号
    .key_in  (key_in), // 按键输入信号
    .down    (down),   // 按下检测输出
    .up      (up),     // 松开检测输出
);

// 模式切换逻辑
always @(posedge clk_100 or negedge rst_n) begin
    if (!rst_n) begin
        mode <= 2'b00; // 复位时模式设为初始模式 0
    end else begin
        if (down[0]) mode <= 2'b00; // 按键0: 切换到模式 0
        else if (down[1]) mode <= 2'b01; // 按键1: 切换到模式 1
        else if (down[2]) mode <= 2'b10; // 按键2: 切换到模式 2
        else if (down[3]) mode <= 2'b11; // 按键3: 切换到模式 3
    end
end

// 写入脉冲生成逻辑
always @(posedge clk_100 or negedge rst_n) begin
    if (!rst_n) begin
        write_pulse <= 1'b0; // 复位时清零
    end else if (down[0]) begin
        write_pulse <= 1'b1; // 按下按键0时产生写入脉冲
    end else begin
        write_pulse <= 1'b0; // 脉冲保持一个周期后清零
    end
end
endmodule

```

2.3 led_proc.v

```

module led_proc (
    input wire      clk,          // 50MHz 时钟输入
    input wire      rst_n,       // 低电平复位信号
    input wire [1:0] mode,       // 模式选择信号

```

```

output wire [7:0] led      // LED 输出
);
reg [7:0] led_pattern; // LED 控制模式寄存器
wire      clk_1; // 1Hz 分频时钟信号

// 实例化 frequency_divider 模块, 将 50MHz 时钟分频到 1Hz
frequency_divider freq_div_inst (
    .clk      (clk),          // 原始时钟信号 50MHz
    .rst_n    (rst_n),       // 低电平复位信号
    .clkbase  (50_000_000),   // 基准时钟频率为 50MHz
    .clkdiv   (1),           // 期望输出频率为 1Hz
    .clkout   (clk_1)        // 输出分频后的 1Hz 时钟信号
);

// 根据 mode 模式和 1Hz 慢时钟信号 clk_1 控制 LED 模式
always @(posedge clk_1 or negedge rst_n) begin
    if (!rst_n) begin
        led_pattern <= 8'b0000_0001; // 复位时默认模式 (初始值)
    end else begin
        case (mode)
            2'b00: led_pattern <= {led_pattern[6:0], led_pattern[7]}; // 左移模式
            2'b01: led_pattern <= {led_pattern[0], led_pattern[7:1]}; // 右移模式
            2'b10: led_pattern <= ~led_pattern; // 闪烁模式
            2'b11: led_pattern <= 8'b1010_1010; // 固定模式
            default: led_pattern <= 8'b0000_0001; // 默认模式为左移
        endcase
    end
end

// 实例化 LED 显示模块, 将 led_pattern 映射到实际 LED 输出
led_display led_display_inst (
    .led_pattern(led_pattern), // 输入的 LED 控制模式
    .led        (led)          // 映射到实际 LED 显示
);
endmodule

```

2.4 seg_proc.v

```

module seg_proc (
    input wire clk, // 50MHz 时钟信号
    input wire rst_n, // 低电平复位信号
    input wire [31:0] seg_number_in, // 输入的数字内容
    input wire [7:0] dp, // 每个位的小数点状态 (1: 显示小数点, 0: 不显示)
    output wire [7:0] dula, // 段选信号
    output wire [7:0] wela // 位选信号
);

wire      clk_1k; // 1kHz 时钟信号
reg [2:0] bits; // 当前显示的数码管位索引 (0 到 7)
reg [3:0] current_num; // 当前显示位的数字内容
reg      current_dp; // 当前显示位的小数点状态

// 分频器实例化, 将 50MHz 时钟分频至 1kHz
frequency_divider u_frequency_divider (
    .clk      (clk),
    .rst_n    (rst_n),
    .clkbase  (50_000_000),
    .clkdiv   (1_000),
    .clkout   (clk_1k)
);

// 根据当前的位选择信号 bits, 选择对应的数码管输入数据和小数点状态
always @(*) begin
    current_num = seg_number_in[(7-bits)*4+:4]; // 选择相应的数字
    current_dp = dp[7-bits]; // 获取小数点状态
end

// 数码管驱动模块实例化
seg_driver u_seg_driver (
    .digit      (current_num),
    .rst_n      (rst_n),
    .current_dp (current_dp),
    .position   (bits),
    .digit_sel  (wela),
    .segment_data(dula)
);

// 动态扫描逻辑, 每 1ms 切换一个显示位

```



```

always @(posedge clk_1k or negedge rst_n) begin
    if (!rst_n) bits <= 3'd0; // 复位时从第 0 位开始显示
    else bits <= (bits == 3'd7) ? 3'd0 : bits + 3'd1; // 切换到下一个显示位
end
endmodule

```

2.5 top.v

```

module top (
    input wire      sys_clk,
    input wire      sys_rst_n,
    input wire [3:0] key,
    output wire [7:0] led,
    output wire      buz,
    output wire [7:0] seg_sel,
    output wire [7:0] seg_data
);

localparam BUZ_ACTIVE_LOW = 1'b1;

// 50MHz -> 100Hz, 给按键消抖
wire clk_100;
frequency_divider u_div_100 (
    .clk      (sys_clk),
    .rst_n    (sys_rst_n),
    .clkbase  (32'd50_000_000),
    .clkdiv   (32'd100),
    .clkout   (clk_100)
);

// 按键消抖后, up 是按下脉冲
wire [3:0] key_press;
key_ctrl u_key (
    .clk_100  (clk_100),
    .rst_n    (sys_rst_n),
    .key_in   (key),
    .down     (),
    .up       (key_press)
);

// 基础练习状态
// S1: 加1
// S2: 减1
// S3: 切换 LED 模式
// S4: 切换蜂鸣器模式 0/1/2/3
reg [3:0] ones;
reg [3:0] tens;
reg [3:0] hundreds;
reg [3:0] thousands;
reg [1:0] led_mode;
reg [1:0] buz_mode;

always @(posedge clk_100 or negedge sys_rst_n) begin
    if (!sys_rst_n) begin
        ones      <= 4'd0;
        tens      <= 4'd0;
        hundreds  <= 4'd0;
        thousands <= 4'd0;
        led_mode  <= 2'd0;
        buz_mode  <= 2'd0;
    end else begin
        if (key_press[0]) begin
            if (ones < 4'd9)
                ones <= ones + 1'd1;
            else begin
                ones <= 4'd0;
                if (tens < 4'd9)
                    tens <= tens + 1'd1;
                else begin
                    tens <= 4'd0;
                    if (hundreds < 4'd9)
                        hundreds <= hundreds + 1'd1;
                    else begin
                        hundreds <= 4'd0;
                        if (thousands < 4'd9)
                            thousands <= thousands + 1'd1;
                        else

```

```

        thousands <= 4'd0;
    end
end
end
end else if (key_press[1]) begin
    if (ones > 4'd0)
        ones <= ones - 1'd1;
    else begin
        ones <= 4'd9;
        if (tens > 4'd0)
            tens <= tens - 1'd1;
        else begin
            tens <= 4'd9;
            if (hundreds > 4'd0)
                hundreds <= hundreds - 1'd1;
            else begin
                hundreds <= 4'd9;
                if (thousands > 4'd0)
                    thousands <= thousands - 1'd1;
                else
                    thousands <= 4'd9;
            end
        end
    end
end
end

if (key_press[2])
    led_mode <= led_mode + 1'b1;

if (key_press[3])
    buz_mode <= buz_mode + 1'b1;
end
end

// LED 练习: S3 切换模式
led_proc u_led_proc (
    .clk (sys_clk),
    .rst_n(sys_rst_n),
    .mode (led_mode),
    .led (led)
);

// 蜂鸣器练习: S4 切换模式
// mode=0 静音, 1/2/3 对应三种音调
wire buz_clk_500;
wire buz_clk_1k;
wire buz_clk_2k;
reg buz_raw;

frequency_divider u_div_buz_500 (
    .clk (sys_clk),
    .rst_n (sys_rst_n),
    .clkbase (32'd50_000_000),
    .clkdiv (32'd500),
    .clkout (buz_clk_500)
);

frequency_divider u_div_buz_1k (
    .clk (sys_clk),
    .rst_n (sys_rst_n),
    .clkbase (32'd50_000_000),
    .clkdiv (32'd1000),
    .clkout (buz_clk_1k)
);

frequency_divider u_div_buz_2k (
    .clk (sys_clk),
    .rst_n (sys_rst_n),
    .clkbase (32'd50_000_000),
    .clkdiv (32'd2000),
    .clkout (buz_clk_2k)
);

always @(*) begin
    case (buz_mode)
        2'd0: buz_raw = 1'b0;
        2'd1: buz_raw = buz_clk_500;
        2'd2: buz_raw = buz_clk_1k;
        default: buz_raw = buz_clk_2k;
    end
end

```

```

        endcase
    end

    assign buz = BUZ_ACTIVE_LOW ? ~buz_raw : buz_raw;

    // 数码管练习显示:
    // COM1-COM4 显示计数值
    // COM5 显示 LED 模式
    // COM6 显示蜂鸣器模式
    // COM7-COM8 熄灭
    seg_proc u_seg_proc (
        .clk            (sys_clk),
        .rst_n          (sys_rst_n),
        .seg_number_in  ({ones, tens, hundreds, thousands, led_mode + 4'd0, buz_mode + 4'd0, 4'd10, 4'd10}),
        .dp             (8'b0000_0000),
        .dula           (seg_data),
        .wela           (seg_sel)
    );

endmodule

```

2.6 top_board.v

```

module top_board (
    input wire    GCLK,
    input wire    RESET,
    input wire    S1,
    input wire    S2,
    input wire    S3,
    input wire    S4,
    output wire    LD1,
    output wire    LD2,
    output wire    LD3,
    output wire    LD4,
    output wire    LD5,
    output wire    LD6,
    output wire    LD7,
    output wire    LD8,
    output wire    BUZ,
    output wire    COM1,
    output wire    COM2,
    output wire    COM3,
    output wire    COM4,
    output wire    COM5,
    output wire    COM6,
    output wire    COM7,
    output wire    COM8,
    output wire    SEGA,
    output wire    SEGB,
    output wire    SEGC,
    output wire    SEGD,
    output wire    SEGE,
    output wire    SEGF,
    output wire    SEGJ,
    output wire    SEGDP
);

    wire    sys_rst_n;
    wire [3:0] key;
    wire [7:0] led;
    wire    buz;
    wire [7:0] seg_sel;
    wire [7:0] seg_data;

    // Board RESET is active-high, while the core uses active-low reset.
    assign sys_rst_n = ~RESET;
    assign key = {S4, S3, S2, S1};

    assign LD1 = led[0];
    assign LD2 = led[1];
    assign LD3 = led[2];
    assign LD4 = led[3];
    assign LD5 = led[4];
    assign LD6 = led[5];
    assign LD7 = led[6];
    assign LD8 = led[7];
    assign BUZ = buz;

```

```

assign COM1 = seg_sel[0];
assign COM2 = seg_sel[1];
assign COM3 = seg_sel[2];
assign COM4 = seg_sel[3];
assign COM5 = seg_sel[4];
assign COM6 = seg_sel[5];
assign COM7 = seg_sel[6];
assign COM8 = seg_sel[7];

assign SEGA = seg_data[0];
assign SEGB = seg_data[1];
assign SEGC = seg_data[2];
assign SEGD = seg_data[3];
assign SEGE = seg_data[4];
assign SEGF = seg_data[5];
assign SEGG = seg_data[6];
assign SEGDP = seg_data[7];

top u_top (
    .sys_clk (GCLK),
    .sys_rst_n(sys_rst_n),
    .key      (key),
    .led      (led),
    .buz      (buz),
    .seg_sel  (seg_sel),
    .seg_data(seg_data)
);

endmodule

```

2.7 uart_parser.v

```

module uart_parser #(
    parameter MAX_LENGTH = 16,          // 最大字符串长度（可配置）
    parameter UART_BPS   = 115200,      // 串口波特率
    parameter CLK_FREQ   = 50_000_000  // 时钟频率
) (
    input wire      clk,                // 时钟信号
    input wire      rst_n,              // 复位信号（低电平有效）
    input wire      rx,                 // 串口接收数据信号
    output wire     parse_done,          // 数据解析完成信号
    output reg [7:0] cmd_signal          // 解析出的命令信号
);

// 串口接收模块输出信号
wire [7:0] po_data; // 串口接收的数据字节
wire      po_flag;  // 串口接收数据完成标志

// 状态机的状态定义
localparam RECEIVE = 2'b00; // 接收数据状态
localparam PARSE   = 2'b01; // 解析数据状态

// 状态机相关信号
reg [1:0] state; // 当前状态
reg [1:0] next_state; // 下一个状态

// 数据缓冲区和计数器
reg [7:0] data_buffer[MAX_LENGTH-1:0]; // 数据缓冲区，存储接收到的数据
reg [4:0] data_count; // 当前缓冲区中的数据计数

// 解析相关信号
reg parse_done_r; // 数据解析完成标志寄存器
reg state_CR; // 检测 '\r' 的状态标志

// ===== 状态机时序逻辑 =====
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= RECEIVE; // 复位时进入接收状态
    end else begin
        state <= next_state; // 更新当前状态
    end
end

// 数据缓冲区的初始化和数据接收逻辑
integer i;
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin

```

```

    data_count <= 0; // 清空接收数据计数器
    state_CR <= 0; // 清除 'r' 状态标志
    for (i = 0; i < MAX_LENGTH; i = i + 1) begin
        data_buffer[i] <= 8'h00; // 将缓冲区初始化为 0
    end
end else begin
    case (state)
        RECEIVE: begin
            if (po_flag) begin // 如果接收到新数据
                if (po_data == 8'h0D) begin // 检测 '\r' (回车符)
                    state_CR <= 1; // 设置回车状态标志
                end else if (state_CR) begin
                    if (po_data == 8'h0A) begin // 检测 '\n' (换行符)
                        state_CR <= 0; // 如果换行符跟在回车符后, 结束本次接收
                    end else begin
                        state_CR <= 0; // 否则清除回车状态标志
                        data_count <= 0; // 重置数据计数器
                    end
                end else if (data_count < MAX_LENGTH) begin
                    data_buffer[data_count] <= po_data; // 存储接收到的数据
                    data_count <= data_count + 1; // 增加计数器
                end
            end
        end
        PARSE: begin
            data_count <= 0; // 清空数据计数器
            state_CR <= 0; // 清除回车状态标志
        end
    endcase
end
end

// 状态机的下一状态逻辑
always @(*) begin
    next_state = state; // 默认保持当前状态
    case (state)
        RECEIVE: begin
            if (po_flag && state_CR && po_data == 8'h0A) begin
                next_state = PARSE; // 如果检测到 '\r\n', 进入解析状态
            end
        end
        PARSE: begin
            if (parse_done_r) begin
                next_state = RECEIVE; // 如果解析完成, 返回接收状态
            end
        end
    endcase
end

// 将解析完成信号输出
assign parse_done = parse_done_r;

// 数据解析逻辑
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        parse_done_r <= 0; // 清除解析完成信号
        cmd_signal <= 8'h00; // 清空命令信号
    end else begin
        case (state)
            PARSE: begin
                parse_done_r <= 1; // 设置解析完成信号
                case (data_buffer[0]) // 根据接收到的第一个字符判断命令
                    "A": cmd_signal <= 8'h01; // 如果接收到 'A', 命令为 0x01
                    "W": cmd_signal <= 8'h00; // 如果接收到 'W', 命令为 0x00
                    default: cmd_signal <= 8'h00; // 其他字符默认为 0x00
                endcase
            end
            default: begin
                parse_done_r <= 0; // 其他状态清除解析完成信号
            end
        endcase
    end
end

// 实例化串口接收模块
uart_rx #(
    .UART_BPS(UART_BPS), // 串口波特率参数
    .CLK_FREQ(CLK_FREQ) // 时钟频率参数
) u_uart_rx (

```

```

        .clk      (clk),      // 输入: 时钟信号
        .rst_n    (rst_n),    // 输入: 复位信号
        .rx       (rx),       // 输入: 串口接收数据
        .po_data  (po_data),   // 输出: 接收到的数据字节
        .po_flag  (po_flag)   // 输出: 接收完成标志
    );
endmodule

```

2.8 uart_string_sender.v

```

module uart_string_sender #(
    parameter MAX_LENGTH = 16,      // 最大字符串长度 (可配置)
    parameter UART_BPS   = 115200, // 串口波特率
    parameter CLK_FREQ   = 50_000_000 // 时钟频率
) (
    input wire      clk,           // 时钟信号
    input wire      rst_n,        // 复位信号 (低电平有效)
    input wire [8*MAX_LENGTH-1:0] string_data, // 输入的字符串数据 (宽总线)
    input wire [3:0] data_length, // 实际需要发送的长度 (动态指定)
    input wire      send_trigger, // 发送触发信号
    output wire     tx,           // 串口发送引脚
    output reg      busy         // 模块忙碌信号 (正在发送数据)
);

// ===== 状态定义 =====
localparam IDLE = 2'b00; // 空闲状态
localparam LOAD = 2'b01; // 装载数据状态
localparam SEND = 2'b10; // 发送数据状态
localparam DONE = 2'b11; // 完成状态

// 状态变量
reg [1:0] current_state, next_state; // 当前状态和下一状态

// ===== 内部信号 =====
reg [3:0] cnt; // 字符计数器, 用于指示发送到第几个字符
reg [7:0] pi_data; // 当前发送的单字节数据
reg      pi_flag; // 当前发送触发信号
wire     tx_done; // 串口发送完成信号

// ===== 状态机: 状态转移逻辑 =====
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        current_state <= IDLE; // 复位时进入空闲状态
    end else begin
        current_state <= next_state; // 状态更新
    end
end

// ===== 状态机: 下一状态逻辑 =====
always @(*) begin
    next_state = current_state; // 默认保持当前状态
    case (current_state)
        IDLE: begin
            if (send_trigger) begin
                next_state = LOAD; // 接收到发送触发信号后, 进入装载状态
            end
        end
        LOAD: begin
            next_state = SEND; // 装载完成后, 进入发送状态
        end
        SEND: begin
            if (tx_done) begin
                if (cnt + 1 == data_length) begin
                    next_state = DONE; // 所有字符发送完成后, 进入完成状态
                end else begin
                    next_state = LOAD; // 否则继续装载下一个字符
                end
            end
        end
        DONE: begin
            next_state = IDLE; // 完成状态结束后, 返回空闲状态
        end
    endcase
end

// ===== 状态机: 状态行为逻辑 =====
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin

```

```

    cnt    <= 0; // 字符计数器复位
    pi_flag <= 0; // 发送触发标志复位
    pi_data <= 8'h00; // 当前发送数据复位
    busy    <= 0; // 空闲标志复位
end else begin
    case (current_state)
        IDLE: begin
            cnt <= 0; // 计数器清零
            pi_flag <= 0; // 清除发送标记
            pi_data <= 8'h00; // 清空发送数据
            busy <= 0; // 标记为空闲
        end
        LOAD: begin
            pi_data <= string_data[(data_length-1-cnt)*8+:8]; // 提取当前要发送的字符
            pi_flag <= 1; // 发送触发标志有效
            busy <= 1; // 标记为忙碌
        end
        SEND: begin
            pi_flag <= 0; // 清除发送触发标志
            if (tx_done) begin
                cnt <= cnt + 1; // 如果发送完成，计数器加1
            end
        end
        DONE: begin
            busy <= 0; // 标记为空闲，发送结束
        end
    endcase
end
end

// ===== 串口发送模块实例化 =====
uart_tx #(
    .UART_BPS(UART_BPS), // 波特率配置
    .CLK_FREQ(CLK_FREQ) // 时钟频率配置
) uart_tx_inst (
    .clk      (clk),      // 时钟信号
    .rst_n    (rst_n),    // 复位信号
    .pi_data  (pi_data),  // 输入：当前发送的单字节数据
    .pi_flag  (pi_flag),  // 输入：发送触发信号
    .tx_done  (tx_done),  // 输出：发送完成标志
    .tx       (tx)        // 输出：串口发送信号
);

endmodule

```